

Rapport TX52

NLP et développement de benchmarks



Présenté à Jaafar Gaber

Le 22 juin 2023

Goudal Victor



Contents

I.	Introduction	4
A.	Expérience et Motivation personnelle.....	4
1.	Expérience passée.....	4
2.	Motivation	5
B.	Objectifs.....	5
C.	Méthodologie et Approche.....	6
II.	Présentation du Natural Language Processing	6
A.	Définition	6
B.	Domaines d'application et cas d'usage.....	7
C.	L'Histoire du NLP	7
D.	Architecture des modèles de DL du NLP.....	7
1.	Text Encoding	8
1.1.	Tokenization.....	8
1.2.	Vectorization.....	9
2.	Anciens modèles de DL.....	11
2.1.	CNN.....	12
2.2.	RNN.....	12
3.	Transformers.....	15
3.1.	Principe d'attention	15
3.2.	Algorithme de Self-Attention.....	16
3.3.	Multi-Head Attention et Masked Multi-Head Attention.....	18
3.4.	Les différentes architectures de Transformers.....	18
3.4.1.	Encoder-Decoder.....	19
3.4.2.	BERT – Encoder – Bidirectional Attention	20
3.4.3.	GPT – Decoder – Unidirectional Attention	20
E.	Récapitulatif.....	21
III.	Nvidia et leurs outils de développement	21
A.	Entraîner un modèle avec TAO Toolkit.....	22
1.	Task Description	23
2.	Download and Preprocess the dataset.....	23
3.	Setting TAO Mounts and paths.....	23
4.	Dataset Convert	24
5.	Training.....	24
6.	Fine-Tuning	24
7.	Evaluation	25
8.	Inference.....	25
9.	Exporter sur Riva.....	25

10.	Récapitulatif.....	26
B.	Déployer un modèle avec Riva	26
1.	Build Riva	26
2.	Deploy Riva	26
3.	Start Riva	26
4.	Run Riva.....	27
C.	Conclusion sur Nvidia	29
IV.	Introduction à HuggingFace et à sa librairie Transformers.....	29
A.	Présentation de HuggingFace et sa librairie Transformers.....	29
B.	Fonctionnalités de base.....	29
1.	Pipeline.....	29
2.	Modèle / Tokenizer / Classification	30
C.	Affinage (Fine-tuning) d'un modèle.....	31
V.	Benchmarking de modèles de NLP	33
A.	Définition et objectif du benchmarking de modèle de NLP	33
B.	Développement de benchmarks	33
1.	Histoire et benchmarks existants	33
2.	MLPerf.....	34
3.	MLCommons.....	34
C.	Données mesurées et librairie python	34
1.	Evaluation de l'entraînement d'un modèle	34
2.	Evaluation d'un modèle.....	35
3.	Application avec sklearn.metrics.....	35
D.	Études de cas sur l'évaluation comparative de modèles NLP à l'aide de HuggingFace.....	37
1.	Présentation de l'étude de cas.....	37
2.	Méthode de résolution.....	37
3.	Résultat d'analyse	38
4.	Ce que l'on peut retenir de cette analyse.....	39
VI.	Conclusion.....	39
A.	Récapitulatif du rapport.....	39
B.	Mon ressenti sur ce projet.....	39
C.	Remerciement.....	40
VII.	Sources	40
A.	Principales sources : vidéos	40
B.	Articles Medium, Datascientest et autres.....	41
C.	Thèses et articles de recherche.....	41
D.	Sites	41

I. Introduction

Dans le cadre d'une matière à projet TX52 proposé par l'Université Technologique de Belfort Montbéliard, j'ai pu découvrir et analyser le domaine du Natural Language Processing (NLP). Ainsi que tester des modèles d'HuggingFace de la librairie Transformer pour pouvoir réaliser des benchmarks.

A. Expérience et Motivation personnelle

Ce projet a pu être réalisé à la suite de nombreuses circonstances car mon passé et mes motivations m'ont finalement permis de réaliser cette matière. En effet, elle est normalement réservée aux étudiants de 4^{ème} et 5^{ème} année. Je vais donc brièvement présenter mon expérience passé dans le monde de l'intelligence artificielle et du Deep Learning, ainsi que mes motivations qui m'ont permis de réaliser ce rapport.

1. Expérience passée

Le semestre dernier, en 1^{er} semestre de branche, je me suis découvert une passion pour le Deep Learning (DL) principalement grâce au **documentaire de Google DeepMind** sur l'intelligence artificiel (IA) développé pour jouer au Go.

En effet, dans cette vidéo « [AlphaGo – The Movie | Full award-winning documentary](#) » disponible sur Youtube, on découvre la réelle puissance et révolution du DL, notamment dans un contexte où le Go fait partie de la culture Coréenne. Ainsi la perte d'un match de Lee Sedol (9^e dan professionnel et l'un des meilleurs joueurs) contre une IA fait trembler le pays et même le monde.

J'ai donc découvert ce nouveau domaine du Deep Learning, premièrement dans le **Reinforcement Learning** (RL ou Apprentissage par Renforcement) où j'ai pu coder sur Python **une IA pour jouer au jeu Snake**. J'ai pu y apprendre toutes les bases du Deep Learning en Python telles que les algorithmes et la librairie Pytorch.

On retrouve ci-dessous (figure 1) une illustration de l'apprentissage du l'IA au jeu du Snake. On remarque qu'elle joue sur plusieurs environnements en même temps et on peut analyser les performances du modèle notamment grâce au score et à la courbe Loss. La fonction Loss évalue l'écart entre les prédictions réalisées par le réseau de neurones et les valeurs réelles des observations utilisées pendant l'apprentissage.

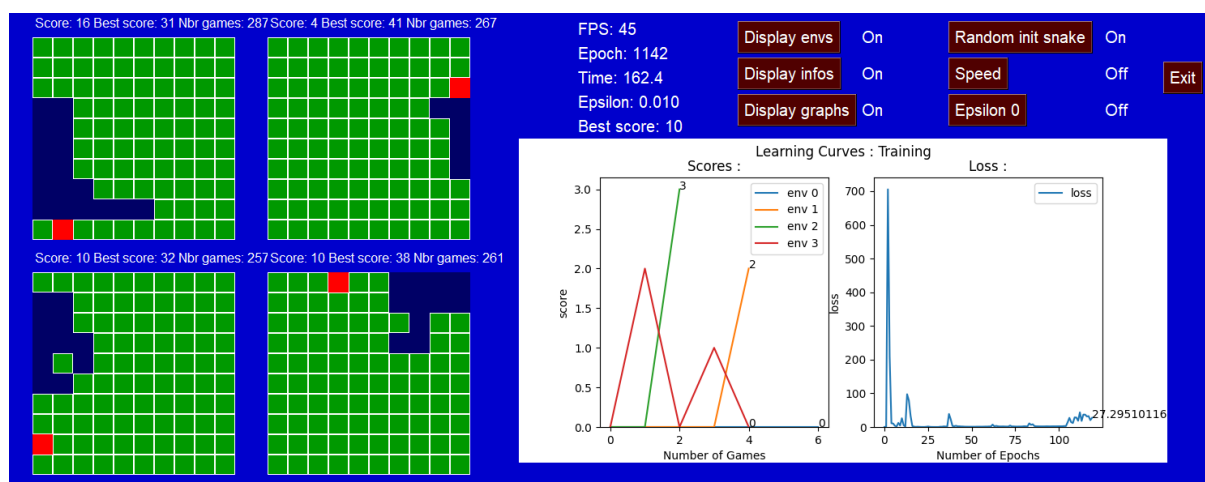


Figure 1 : Interface graphique lors de l'entraînement du Snake. Source : Auteur.

Tout en restant dans le DL, je me suis ensuite intéressé à la vision virtuelle en réalisant des IA permettant la **classification d'image** ou la **génération d'image**. Puis, en utilisant un modèle pré-entraîné, j'ai pu réaliser de la détection et **reconnaissance d'objet** sur des images.

Ainsi, de par la passion que je porte pour ce domaine de l'intelligence artificielle, j'ai pu acquérir de solide base en DL pour pouvoir maintenant étudier le domaine du langage humain.

2. Motivation

Suite à ce 1^{er} semestre, durant l'inter semestre, grâce au partage d'une offre de stage de mon professeur de Parallel Computing, Mr Jaafar Gaber, et de mon expérience passée, j'ai pu obtenir un stage de 4^{ème} année au Centre National de la Recherche Scientifique (CNRS), plus exactement à l'Institut du Développement et des Ressources en Informatique Scientifique (IDRIS) à Paris. L'IDRIS est le centre national du CNRS pour le calcul numérique intensif de très haute performance (HPC pour High Performance Computing) et l'intelligence artificielle (IA), au service des communautés scientifiques de la recherche publique ou privée. Ils possèdent notamment le plus grand Supercalculateur de France.

Ainsi, le sujet étant « L'intelligence artificielle distribuée sur supercalculateur », je devrais notamment **tester les outils récents** d'intégration et d'optimisation de système de DL à grande échelle et **développer des benchmarks**.

Donc j'ai choisi de faire ce projet pour pouvoir agrandir mes connaissances dans le DL, me préparer au stage tout en optimisant mon temps d'étude car j'ai pu remplacer une matière standard de mon cursus par cette matière à projet.

B. Objectifs

Ce rapport a plusieurs objectifs, autant personnel que pour le lecteur.

Personnellement, ce rapport m'aura permis d'en apprendre beaucoup plus sur le domaine qui révolutionne aujourd'hui le monde, le NLP, j'ai donc pu me préparer significativement pour le stage de 4^{ème} année.

Ce rapport a aussi pour objectif de faire découvrir et comprendre au lecteur cette nouvelle forme de technologie qui est l'intelligence artificielle, qui fait maintenant partie intégrante de notre vie, comme par exemple chatGPT qui est un modèle du Deep Learning de traitement du langage humain.

C. Méthodologie et Approche

Durant toute mon expérimentation, j'ai dû exécuter du code python ainsi j'ai utilisé 2 environnements en fonction de la puissance de calcul nécessaire :

- Pour la découverte du HuggingFace et des transformers, je suis resté sur ma machine personnelle avec l'outil Visual Studio Code. Je possède un pc avec un Intel Core I5 et une carte graphique Nvidia GTX 1650. Ainsi j'ai pu utiliser la librairie cuda pour utiliser mon GPU.
- Ensuite, pour l'entraînement de transformers, je suis passé sur Google Colab Pro pour pouvoir avoir accès à des A100 et donc avoir une immense puissance de calcul. Tous mes modèles sont donc déplacés sur le GPU grâce à cuda pour profiter de la pleine puissance des GPU. Et toutes les données sont ensuite stockées sur mon Google Drive.



II. Présentation du Natural Language Processing

Avec le développement et la popularisation de l'intelligence artificielle dans l'assistance au quotidien, la communication entre le robot ou entité numérique et l'utilisateur ne peut plus se faire dans un langage codé ou sous forme de programme informatique avec des lignes de codes. Ainsi pour communiquer dans le langage naturel de l'humanité, le domaine de la science du Traitement Automatique du Langage Naturel est créé.

A. Définition

Le Natural Language Processing (NLP) ou Traitement Automatisé du Langage Naturel (TALN) est une forme d'intelligence artificielle. Il permet aux ordinateurs d'analyser, de comprendre le langage humain pour pouvoir ensuite le traiter en tant que donnée pure. Elle peut ensuite manipuler ces données pour générer une réponse et donc créer une conversation naturelle à l'homme.

On peut distinguer 2 sous-parties dans ce domaine qui sont :

- Le **Natural Language Understanding** (NLU) ou Compréhension du Langage Naturel. Il permet de comprendre en profondeur les échanges et les données, pour identifier les idées et intentions des paroles ou des écrits humains.
- Le **Natural Language Generation** (NLG) ou Génération du Langage Naturel. Il permet de créer et générer automatiquement des échanges dans une langue demandée, grâce à l'intelligence artificielle. Ainsi les entreprises peuvent automatiser cette relation au client car l'IA peut convertir les données numériques en texte compréhensible par l'humain.

B. Domaines d'application et cas d'usage

Le NLP possède de nombreux domaines d'application dont les entreprises se sont déjà emparées pour automatiser cette communication avec l'utilisateur. On remarque principalement 2 applications très populaire :

- La **génération de texte** : cela consiste à générer du texte en fonction de donnée ou comme dans les chatbots en fonction du texte envoyé par l'utilisateur. Ainsi le NLP est utilisé dans de nombreuses applications telles que les applications de traduction comme Google Translate, les assistants vocaux comme Apple Siri, Microsoft Cortana et Amazon Alexa, et, les chatbots comme le plus connu ChatGPT.
- La **correction automatique** : cela permet de détecter et donc pouvoir corriger les fautes d'orthographe, d'accords voire maintenant les erreurs de syntaxe ou de sens. De nombreux éditeurs de texte se sont munis de cette technologie en tant que correcteur automatique. On peut notamment citer Word de Microsoft ou Grammarly.

On peut aussi citer d'autres domaines d'application, moins connu du grand public mais tout aussi important :

- La **classification de texte** : cela permet de trier des textes entre différentes catégories de manière automatique. Elle est principalement utilisée dans nos messagerie ou mailbox pour déterminer si un mail est un spam ou non.
- La **reconnaissance de caractères** : cela permet d'identifier dans un texte des éléments très précis comme des dates, des chiffres clés ou des informations importantes. Elle est déjà utilisée dans la lecture automatique de reçus, de factures, de chèques, etc...
- Le **résumé de texte** : cela permet, de par son nom, de synthétiser un texte en identifiant les idées importantes pour pouvoir le reformuler dans un texte plus court, précis et fluide.
- L'**inférence du langage Naturel (NLI)** ou Natural Language Inference : cela permet de savoir si 2 phrases, une hypothèse et une prémisse, sont similaires, contradictoire ou neutre. Par exemple, les phrases « il fait un temps terrible dehors » et « ce temps me donne envie l'aller faire une promenade dehors » sont contradictoires.

C. L'Histoire du NLP

L'étude du NLP a commencé dans les années 1950

- 1950 : Alan Turing, dans son article intitulé « Computing Machinery and Intelligence », a proposé le « test de Turing ». Ce test est conçu pour mesurer à quelle point le comportement d'une machine, intelligence artificielle est similaire à celui du comportement humain. Ainsi de nombreux chercheurs se sont penchés sur la question du langage humain qui est l'une des caractères principaux de notre espèce.
- 1953 : En collaboration avec l'entreprise d'informatique IBM, l'université GeorgeTown a proposé le premier traducteur électronique. Cette algorithmes permet de traduire du Russe à l'anglais.
- 2000 : Dans les années 2000, avec l'amélioration de la puissance de calcul des machines, les modèles de traitement du langage naturel basés sur les réseaux de neurones ont commencé à émerger telle que les Convolutional Neural Network (CNN), Recurrent Neural Network (RNN).
- 2017 : Les modèles Transformers ont révolutionné le traitement du langage avec des performances inédites et qui dépassent tout algorithmes ou modèles précédents.

D. Architecture des modèles de DL du NLP

Les algorithmes basés sur des réseaux de neurones possèdent toujours une structure similaire. En effet, ils sont composés d'un encodeur qui permet de convertir le texte en vecteur. Il est lui-même composé d'un Tokenizer qui sépare les phrases de plusieurs manières et d'un convertisseur qui convertie les tokens en vecteurs. Ensuite ces vecteurs sont traités par un modèle. Puis un décodeur convertie les vecteurs renvoyés par ce modèle en texte.

1. Text Encoding

Il existe notamment différents encodeurs avec différents tokenizers et convertisseurs.

1.1. Tokenization

Le Tokenizer permet de séquencer un texte. Il existe différentes méthodes de séquençage qui ont chacun leur avantage et leur inconvénient.

Le plus logique est le « **Word-based tokenizer** ». En effet, il consiste à simplement séparer les phrases au niveau des espaces ou de la ponctuation.

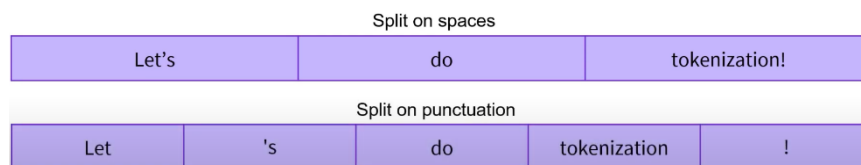


Figure 2 : Word-base tokenizer. Source : Vidéo HuggingFace sur Tokenizer.

L'avantage est que chaque token possède une signification unique. Cependant cet avantage peut être aussi un inconvénient. En effet, par exemple, les mots « chien » et « chiens » ont globalement le même sens même si l'un est au pluriel, mais pour le modèle, ce sont 2 mots complètement différents. L'autre problème que ça implique est que le « Dictionnaire des mots » peut être très grand, ce qui demande alors un modèle plus puissant. Une solution qui avait été envisagée était donc de spécialiser le modèle à un dictionnaire plus restreint. Cependant, si une phrase possédait un mot qui n'était pas dans le dictionnaire, alors le modèle ne pouvait pas interpréter cette phrase.

Ensuite, à l'opposé du « Word-based tokenizer », le « **Character-based tokenizer** » est créé pour résoudre les inconvénients du précédent tokenizer.

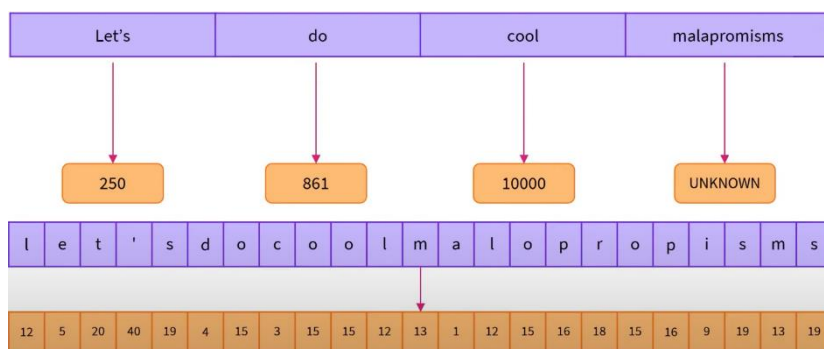


Figure 3 : Character-base tokenizer. Source : Vidéo HuggingFace sur tokenizer.

En effet l'avantage de ce tokenizer est qu'il possède un vocabulaire très petit d'environ 256 contre plusieurs milliers voire millions pour le « Word-based tokenizer ». Mais chaque token ne possède indépendamment

aucune signification, ce qui rend la compréhension beaucoup plus compliqué puisqu'un caractère peut avoir des significations opposées en fonction des tokens (caractères) voisins.

Ainsi les plus grands modèles de NLP actuels tels que BERT ou GPT possède des « **Subword-based tokenizers** ». Il est assez similaire au « Word-based tokenizer » mais ensuite les mots sont, de nouveau, découpés en sous-mots.

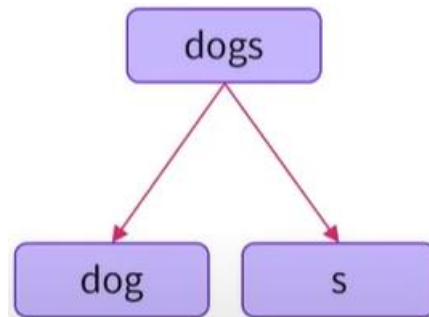


Figure 4 : Subword-base tokenizer. Source : Vidéo HuggingFace sur tokenizer.

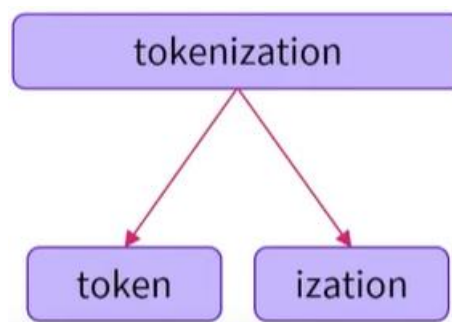


Figure 5 : Subword-base tokenizer. Source : Vidéo HuggingFace sur tokenizer.

Cette méthode permet donc de trouver un compromis entre le « Word-based tokenizer » et le « Character-based tokenizer » en ne récupérant principalement que les avantages. En effet, le vocabulaire est grandement réduit car par exemple, le token « s » signifie directement que le mot précédent est au pluriel, ce qui élimine les tokens « chiens », « chats », « poissons », etc... Et cette méthode permet aussi donc de garder des tokens (mots) qui possède une signification à l'opposé du « Character-based tokenizer ».

1.2. Vectorization

Une 2nd partie de l'encoder est, comme dit précédemment, la **vectorisation du texte en numérique**. Elle permet au modèle de pouvoir lire et récupérer ces données. Nous verrons, ici, 4 méthodes : « From text to tensor », « One Hot encoding », « One Hot vector », « Embedding ». Dans les exemples ci-dessous, nous utiliserons un word-based tokenizer.

La 1^{ère} méthode « **From text to tensor** » est la plus simple mais la moins efficace. Elle consiste à associer à chaque mot du dictionnaire un nombre entier.

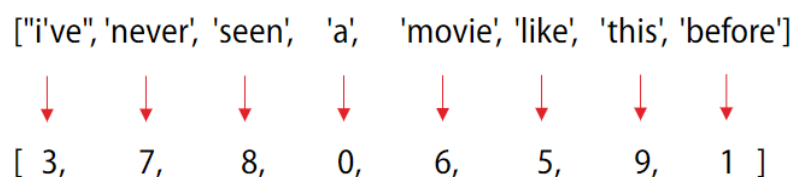


Figure 6 : Text-to-Vector. Source : Vidéo Fidle CNRS sur Données creuses.

Cependant, le problème principal est que les valeurs associées à chaque mot n'ont pas sens et ne sont pas liées les unes-les-autres. En effet, les mots aux valeurs proches telles que « never » et « seen », avec respectivement 7 et 8, ne sont pas plus liés qu'avec « before » qui possède une valeur, plus éloignée, de 1.

Ainsi une 2nd méthode, appelée « **One Hot encoding** », permet de lier les mots du dictionnaire entre eux en associant à chaque mot du dictionnaire une dimension.

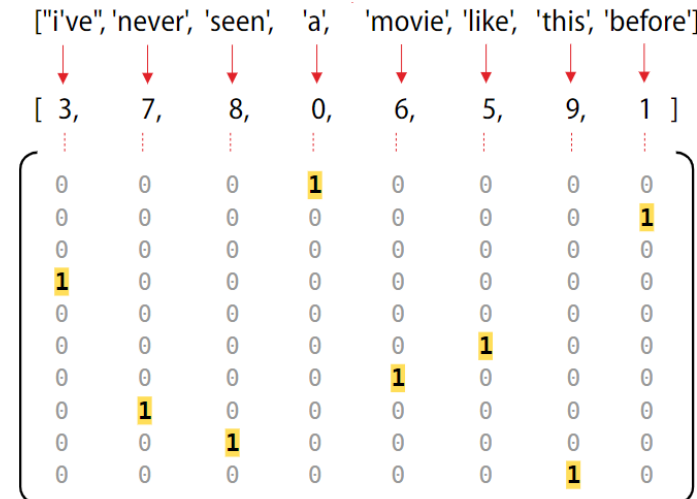


Figure 7 : One Hot Encoding. Source : Vidéo Fidle CNRS sur Données creuses.

Il possède déjà plusieurs avantages par rapport au précédent. En effet, la conversion en vecteur binaire permet une meilleure comparaison et manipulation des mots. Aussi comme la dimension du One Hot vecteur encodé correspond à un mot spécifique du vocabulaire, ça facilite l'interprétation de la signification du vecteur et la visualisation des relations entre les mots. Ce qui permet d'avoir de bon résultat par la suite mais il possède cependant certaines limites.

Le principal problème de cette méthode est sa taille. Par exemple, pour un dictionnaire de 80 000 mots et une phrase en entrée de l'encoder de 300 mots, on obtiendrait une matrice de 24 000 000 paramètres pour le modèle, ce qui demanderait un très gros modèle pour supporter autant de paramètre en entrée.

Pour cela, une méthode envisagée, dans le cas où le nombre de paramètre du modèle serait trop élevé, est d'additionner les vecteurs de la matrice pour n'en former plus qu'un. Elle s'appelle « **One Hot Vector** ». On passe alors, dans l'exemple du dessus, d'une matrice de 24 000 000 paramètres à un vecteur de 80 000 paramètres qui ne tient donc pas compte de la taille du texte en entrée. Mais l'inconvénient est que nous perdons l'ordre des mots donc cette méthode réduit la précision des informations.

Ainsi, la méthode de l'**embedding**, popularisée assez tardivement à partir de 2013, permet d'optimiser la taille des vecteurs tout en gardant toutes les informations. En effet, en reprenant l'exemple du dessus, cette méthode consiste à convertir les vecteurs creux du « One Hot Encoding » de 80 000 mots en des vecteurs

denses de 200 valeurs. On se place alors dans un monde où chaque mot est placé en fonction des autres. On peut le voir dans l'illustration ci-dessous très parlante.

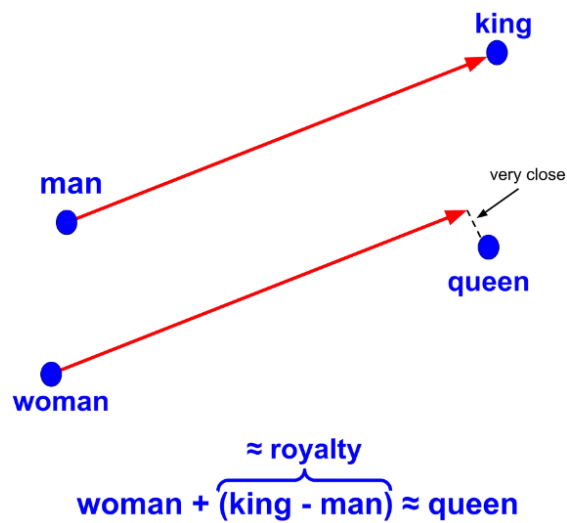


Figure 8 : Embedding. Source : Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Aurélien Géron.

Grâce à cette méthode, on peut facilement visualiser le sens des mots et ça permet au modèle de facilement réaliser des opérations sur les mots. C'est aujourd'hui la méthode la plus utilisée, elle est notamment utilisée avec les modèles les plus puissants (Transformers). Nous n'entrerons pas dans les détails mais il existe 2 techniques d'embedding : le **contextual embedding** implémenté par Keras embedding et le **global embedding** implémenté par CBOW, SG, GloVe, etc...

2. Anciens modèles de DL

Avant l'arrivée du Transformer qui a révolutionné le domaine du NLP en 2017 dans un article intitulé « Attention is All You Need » de Vaswani et al, on faisait avec des modèles de type Convolutional Neural Network (CNN) et plus tard le Recurrent Neural Network (RNN) et RNN bidirectionnelle.

De par la puissance des Transformers, tous les plus gros modèles, de nos jours, en sont équipés. Le graph ci-dessous affiche le score de plusieurs modèles testés avec GLUE. GLUE est un ensemble de dataset qui permet de les tester sur différents domaines du NLP telles que les Question/Réponse, les Résumés, l'analyse de sentiment, etc.... Et on voit bien que les modèles les plus performants sont des Transformers.

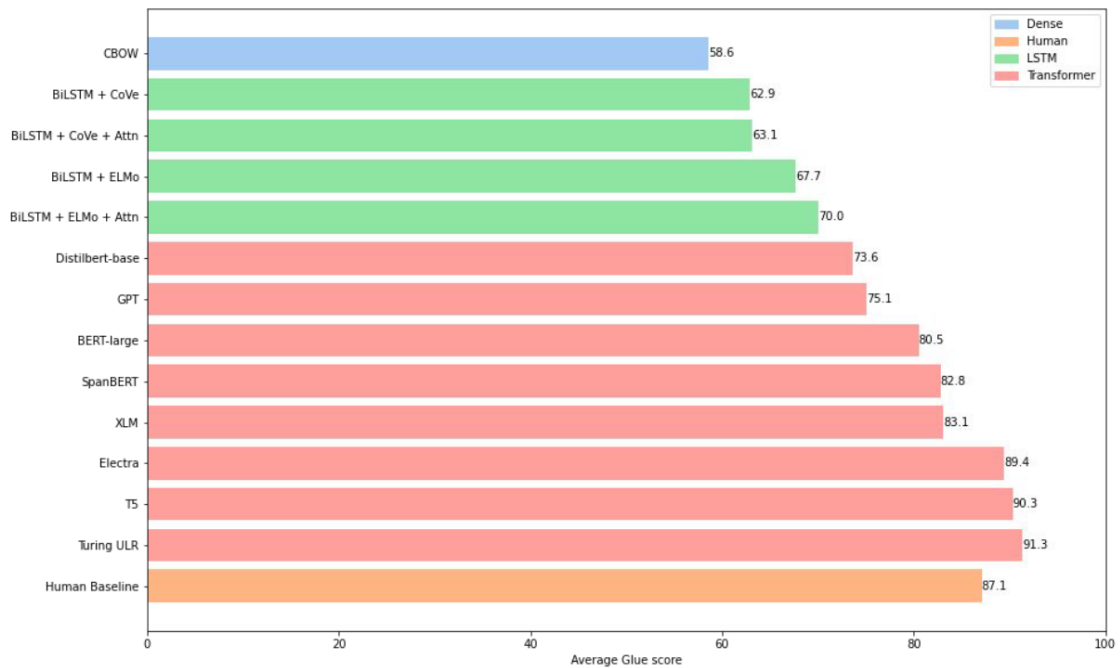


Figure 9 : Benchmark NLP models. Source : Vidéo Fidle sur Transformers.

2.1. CNN

Pour rappel, les CNN sont très utilisés dans la vision virtuelle et le traitement d'image car il possède des convolutional layers, qui appliques des filtres qui apprennent à reconnaître des éléments intéressants. Par exemple, chaque filtre peut se spécialiser sur une partie de la tête pour reconnaître un visage dans une image tel que le nez, les yeux, la bouche, etc... Ces filtres scan l'image un peu comme une fenêtre qui se déplace sur cette dernière. Dans le cas du NLP, cette notion de fenêtre permet d'analyser des mots avec leurs voisins, ainsi on obtient plus d'informations sur les mots en question.

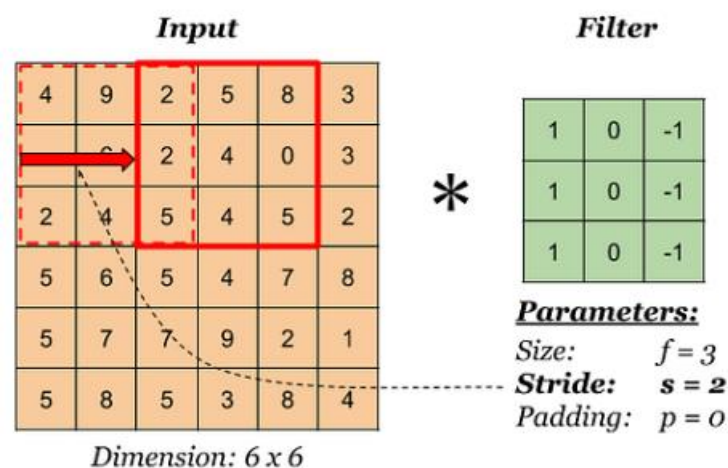


Figure 10 : CNN démonstration. Source : Indoml

Ainsi pour certaines tâches, le CNN peut être assez efficace car chaque mot est étudié dans son contexte. Cependant cette notion de fenêtre possède un inconvénient. En effet, lorsque le texte en entrée est trop long, cette fenêtre n'est pas assez large pour étudier des mots en fonctions de leurs voisins éloignés.

2.2. RNN

Avant l'arrivée des Transformers, la structure la plus optimale était les « **Recurrent Neural Network** ». Ils sont super efficaces dans le traitement de donnée continue comme les signaux sonores, électriques et dans le NLP, les suites de mots. En effet, comme on peut le voir sur l'image ci-dessous, le modèle est rappelé en boucle pour chaque input, ce qui permet de faire varier la taille des valeurs en entrée et de pouvoir étudier de longs vecteurs de données. Dans notre cas où l'on a un « word-based tokenizer », chacun nouvelle donnée en entrée correspond à un seul mot ou ponctuation.

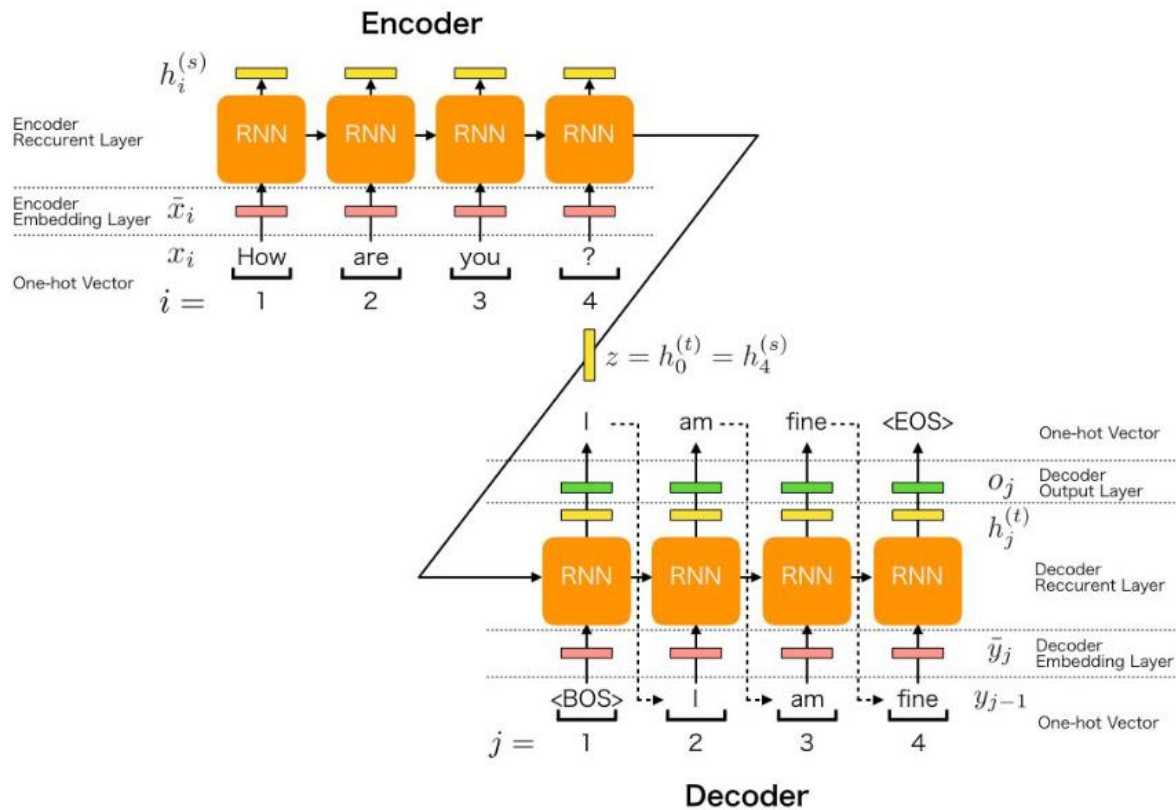
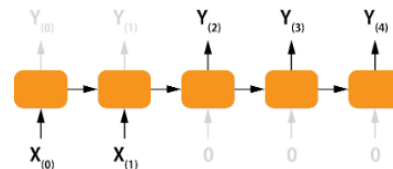


Figure 11 : Encoder-Decoder. Source : <https://docs.chainer.org/en/v7.8.0/examples/seq2seq.html>

Dans l'exemple ci-dessus, le modèle est sous forme d'encoder / décodeur. L'encoder prend en entrée chaque mot et renvoie ensuite un vecteur au décodeur avec la signification de la phrase. Ensuite en fonction de la mission du modèle tel que la traduction ou la génération d'une conversation, le décodeur va prendre en entrée ce vecteur et le token <BOS> pour « Beginning Of Sentence » pour générer la suite de texte en fonction du mot précédent et du vecteur. Le décodeur fini par token <EOS> pour « End Of Sentence » pour finir l'exécution.

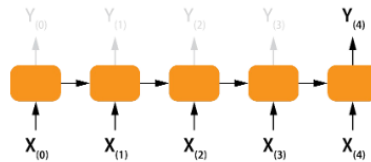
Ainsi cette structure est très polyvalente et permet de s'adapter au problème posé. On a déjà vu, ci-dessous, le cas où l'on prend entrée une phrase et on obtient une phrase en sortie avec l'encoder / décodeur. Mais on peut aussi ne renvoyer qu'une valeur pour faire de la catégorisation tel que l'analyse de sentiment ou l'analyse de SPAM.



Encoder-decoder

Example : Language Translation

Figure 12 : Encoder-Decoder. Source : Vidéo Fidle sur RNN.



Serie to vector

Example : Sentiment analysis

Figure 13 : Serie to Vector. Source : Vidéo Fidle sur RNN.

Pour obtenir de bon résultat, on utilise la meilleure version de RNN qui est le **Long Short-Term Memory (LSTM)**. Il possède cette fois-ci 2 vecteurs en entrée entre chaque itération : un état à court terme qui permet dans le cas du NLP, de pouvoir étudier un mot en fonction de ses proches voisins et un état à long terme pour se souvenir du contexte global de la phrase. Ci-dessous, on peut voir la composition d'une cellule LSTM.

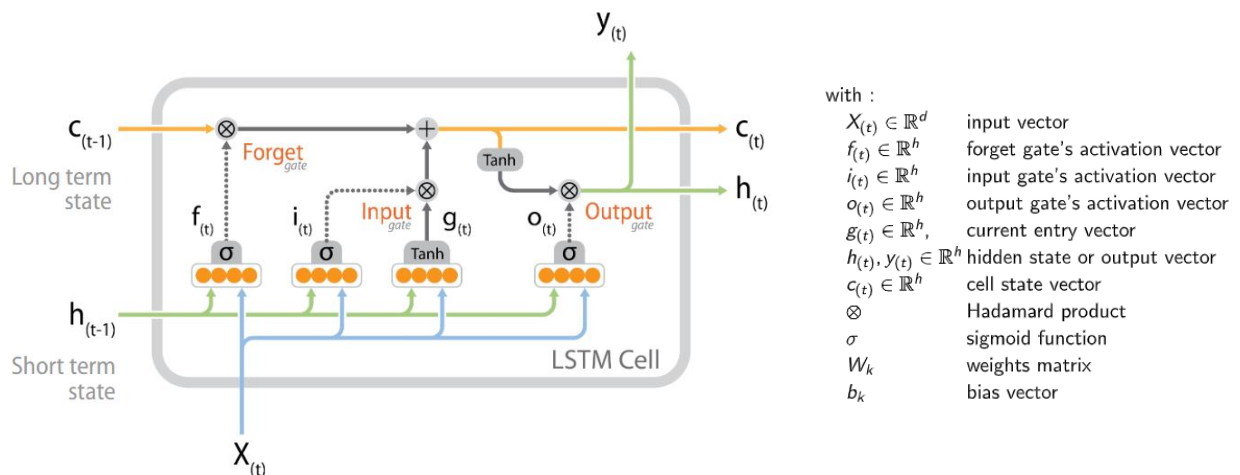


Figure 14 : Cellule de LSTM. Source : Vidéo Fidle sur RNN

On n'entrera pas dans les détails du fonctionnement du LSTM mais on retrouve bien les inputs et outputs vu précédemment tel que $X(t)$ et $Y(t)$ les mots en entrée et sortie du modèle et $C()$ et $H()$ les vecteurs qui permettent de lier les RNN en eux.

Pour conclure sur les RNN, cette structure était assez performante pour des données assez réduites. Cependant elle possède quelques inconvénients. En effet, sachant que les mots sont étudiés les uns après les autres, plus le texte est long, plus l'impact des premiers mots est dilué. Ensuite sachant que le modèle a besoin de la sortie du mot précédent pour fonctionner, on ne peut pas le paralléliser sur plusieurs GPU, ce qui est hyper contraignant lorsque les modèles deviennent de plus en plus gros et que nous voulons étudier de très grandes quantités de données.

3. Transformers

Ainsi sont apparus les Transformers qui permettent de répondre à toutes les problématiques des CNN et RNN. En effet, ils permettent de traiter des séquences comme le RNN tout en étant plus efficace et pouvant être distribuer sur plusieurs GPU. Ainsi, avec cette parallélisation, on peut entrainer ces modèles avec des quantités astronomiques de données. De plus, les Transformers ont la particularité de pouvoir être pré-entraîné, notamment en apprenant déjà à comprendre et générer dans texte dans une langue et il peut être ensuite « fine tune » ou affiner pour pouvoir être adapter à des problèmes spécifiques. Ce qui permet d'économiser du temps et de l'énergie en évitant l'entraînement de la compréhension du langage.

La structure de ce modèle repose sur le principe **d'Attention**.

3.1. Principe d'attention

Le mécanisme d'attention permet au modèle de se concentrer sur certaines parties de la séquence en entrée en fonction de l'importance de chaque mot. En d'autres termes, il permet de créer des connexion / corrélation entre certains mots de la phrase.

Dans l'exemple ci-dessous, dans la phrase « The boy is holding a blue ball », lors de l'analyse par le modèle, la balle va être fortement liée à la couleur bleue qui caractérise la balle et de même pour tient qui exprime l'état de la balle. A l'opposé, la couleur bleue sera peu liée au garçon.



Figure 15 : Attention. Source : Transformers Explained Visually (Part 1) : Overview of Fonctionality de Ketan Doshi

Ainsi ce mécanisme d'attention va étudier le rapprochement de chaque mot avec l'entière des mots de la phrase en entrée. Ce principe permet, par la même occasion, de donner du sens à des mots pouvant être ambigus. Prenons pour exemple les deux phrases ci-dessous.

- The *cat* drank the milk because it was hungry.
- The cat drank the *milk* because it was sweet.

Figure 16 : Attention. Source : Transformers Explained Visually (Part 1) : Overview of Fonctionality de Ketan Doshi

Dans la 1^{ère} phrase, le mot « it » se réfère au chat car le lait ne peut pas avoir faim. A l'opposé, dans la 2nd phrase, le mot « it » se réfère au lait car le lait a plus de chance d'être sucré. Ainsi quand le modèle traite le mot « it », le mécanisme d'attention permet de lui donner plus d'information par rapport à son sens. Ce qui permet d'associer « it » avec le bon mot et ainsi lui donner plus de sens qu'un simple pronom personnel.



Figure 17 : Attention porté sur le mot it. Source : Transformers Explained Visually (Part 1) : Overview of Fonctionality de Ketan Doshi

En utilisant un dégradé de rouge, on peut observer le degré de connexion de « it » avec chaque mot de la phrase. On remarque bien que, dans le cas où il a faim, « it » possède sa plus forte connexion avec le chat, et inversement dans le cas de la 2nd phrase.

Ainsi, dans le rapport de recherche « Attention is all you need », un algorithme, appelé **Self-Attention**, est créé en se basant sur ce principe.

3.2. Algorithme de Self-Attention

Cet algorithme est annoncé pour la 1^{ère} fois dans « Attention is All You Need ». Il se compose de plusieurs étapes détaillées ci-dessous :

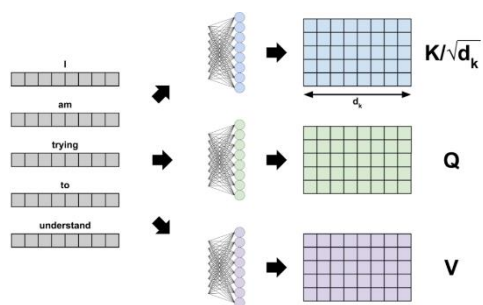


Figure 18 : KQV. Source : Vidéo Fidèle sur Transformers

Après le pre-processing et l'embedding, on obtient une matrice étant une liste de vecteur représentant chaque mot de la phrase. Cette matrice est ensuite passée dans 3 couches de neurones denses différentes pour obtenir 3 nouvelles matrices K (Key), Q (Query) et V (Value). Chaque matrice possède sa spécialité propre que l'algorithme cherche à obtenir en modifiant ces 3 réseaux denses.

Ce concept de K (Key), Q (Query) et V (Value) provient du même article. **Query** est la requête ou le texte en entrée, c'est donc la liste des vecteurs de chaque mot. **Key** représente les informations utilisées pour récupérer le

contexte pertinent de la requête, il permet en quelque sorte d'indexer les informations importantes. Et value contient l'information contenue dans chaque mot et donc dans le texte.

Pour illustrer cela, prenons un exemple : quand tu recherches des vidéos sur Youtube, tu vas rentrer les informations voulues dans la barre de recherche (*Query*). Ces informations vont être traitées pour récupérer que les informations importantes grâce aux mots-clés (*Key*) qui dans notre cas sont le titre des vidéos, la

description, l'auteur, etc. Ainsi YouTube va t'afficher le contenu le plus pertinent (*Value*) en fonction de la requête et de sa base de données.

Ensuite, une 2nd étape consiste à réaliser un produit scalaire entre chaque mot pour obtenir la matrice d'attention et donc pour mettre en avant les connexions et les informations importantes. Le Softmax permet d'obtenir des probabilités dans l'intervalle [0,1], ce qui optimise l'information.

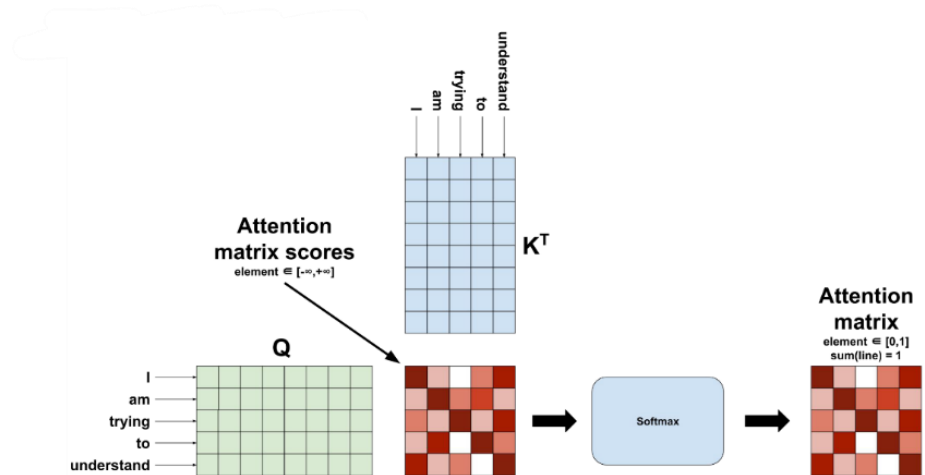


Figure 19 : Matrice d'Attention. Source : Vidéo Fidle sur Transformers

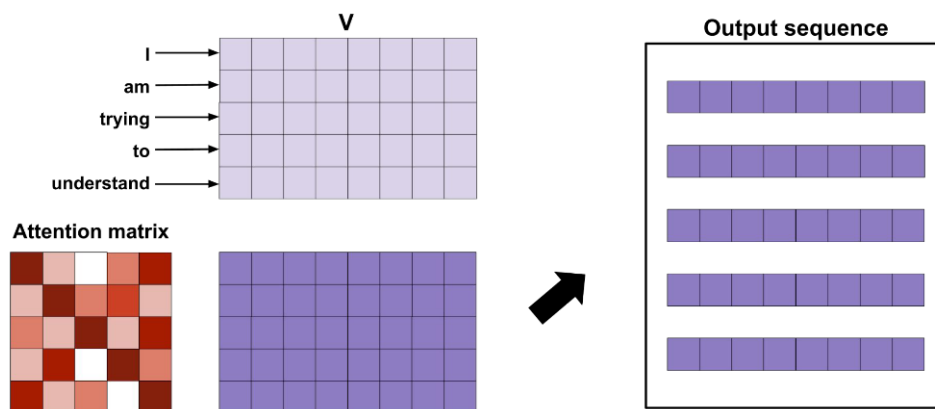


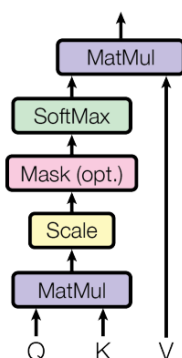
Figure 20 : Matrice d'Attention x V. Source : Vidéo Fidle sur Transformers

Pour finir, la matrice Value est multipliée par la matrice d'attention. Et chaque vecteur de sortie possède un mix de tous les mots mais avec des coefficients différents.

On peut résumer cela avec, ci-dessous, le schéma de l'article

« Attention Is All You Need » et sa formule avec d_k la dimension de la clé. Ensuite une amélioration du self-attention est le Multi-Head Attention. C'est notamment le composant far du modèle Transformer. Une variante

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Figure 21 : Schéma algorithme et formule du Self-Attention. Source : Attention Is All You Need de Vaswani A. et al.

est le Masked Multi-Head Attention qui permet d'entraîner le modèle à la génération de texte en cachant des parties de la séquence.

3.3. Multi-Head Attention et Masked Multi-Head Attention

Le **Multi-Head Attention** est une extension du Self-Attention qui améliore la capacité du modèle à se concentrer sur différents aspects de la séquence d'entrée. Elle divise les clés, les requêtes et les valeurs en plusieurs têtes, et chaque tête effectue un calcul d'attention distinct. Les résultats des différentes têtes sont ensuite concaténés et transformés linéairement pour obtenir le résultat final. L'attention à têtes multiples permet au modèle de s'intéresser simultanément à différentes parties de l'entrée et de saisir différents types de relations ou de modèles.

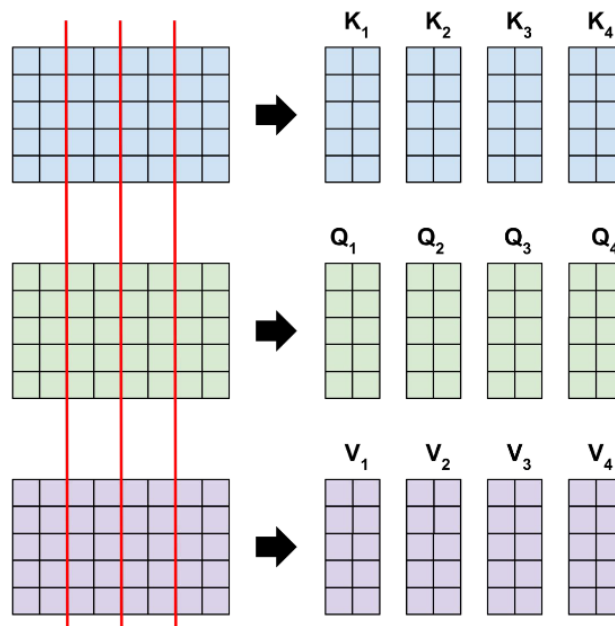


Figure 22 : Multi-Head Attention. Source : Vidéo Fidle sur Transformers.

En reprenant les illustrations de l'explication de l'algorithme de Self-Attention, on remarque sur l'image ci-dessus que le Multi-Head Attention consiste juste à diviser les matrices K, Q et V et ensuite les concaténer à la fin. Le nombre h de division (dans l'exemple ci-dessus, $h = 4$) est un **Hyperparamètre**, il dépend donc du problème demandé par le programmeur du modèle.

Une variante aussi utilisée du Multi-Head Attention est le **Masked Multi-Head Attention** qui consiste aussi à masquer des informations. Ainsi le modèle s'entraîne à prédire les informations qui vont suivre.

Pour conclure, l'Attention est la base des Transformers notamment avec les composants Multi-Head Attention et Masked Multi-Head Attention. Ensuite nous allons voir les structures des 3 Transformers les plus connus et comment ils utilisent ces composants.

3.4. Les différentes architectures de Transformers

Les structures des Transformers possèdent souvent les mêmes composants. Nous avons précédemment vu l'Embedding, le Multi-Head Attention et le Masked Multi-Head Attention. Ils possèdent aussi d'autres éléments :

- **Feed Forward Layer** : Une couche d'alimentation directe est un type de couche de réseau neuronal qui applique une transformation linéaire sur chaque position de la séquence indépendamment. Elle se compose de deux transformations linéaires avec une fonction d'activation non linéaire entre les deux.

L'objectif de la couche d'alimentation directe est d'introduire des non-linéarités et d'augmenter la capacité d'expression du modèle. Elle permet au modèle de capturer des relations et des motifs complexes au sein de la séquence.

- **Add and Normalize** : L'étape « Add and Normalize » est appliquée après chaque sous-couche à l'intérieur d'un bloc Transformer. Elle consiste à ajouter la sortie de la sous-couche à l'entrée, suivi d'une opération de normalisation de couche. Le but de cette étape est d'atténuer les problèmes de disparition ou d'explosion du gradient lors de l'entraînement et de stabiliser le processus d'apprentissage. En ajoutant la connexion résiduelle (la sortie de la sous-couche) à l'entrée et en appliquant une normalisation de couche, le modèle peut propager et mettre à jour plus efficacement les informations au sein de la séquence.

Elle joue un rôle crucial dans la facilitation de la propagation de l'information à travers l'architecture des Transformers. Elle permet au modèle de conserver l'entrée d'origine tout en incorporant les informations traitées de chaque sous-couche, ce qui lui permet de capturer à la fois les dépendances locales et globales au sein de la séquence.

En utilisant ces composants, il existe de nombreuses structures différentes. Les Transformers les plus imposants sont BERT, GPT dont nous connaissons la version 3.5 et 4 avec ChatGPT, XLNet, T5 qui est un Text-to-Text Transfert Transformer. Les structures du Transformer de base, BERT et GPT vont être détaillés par la suite pour voir leurs caractéristiques.

3.4.1. Encoder-Decoder

Ci-dessous, se trouve la structure du Transformer de la base annoncée dans l'article « Attention Is All You Need » de 2017. Il est composé d'un stack d'Encoder et ensuite d'un autre de Décodeur. Ce Transformer de base est efficace pour faire des connections entre des mots très éloignés. Il a été entraîné sur des datasets de traduction Anglais-Allemand et Anglais-Français, ainsi il était spécialisé dans la traduction de texte.

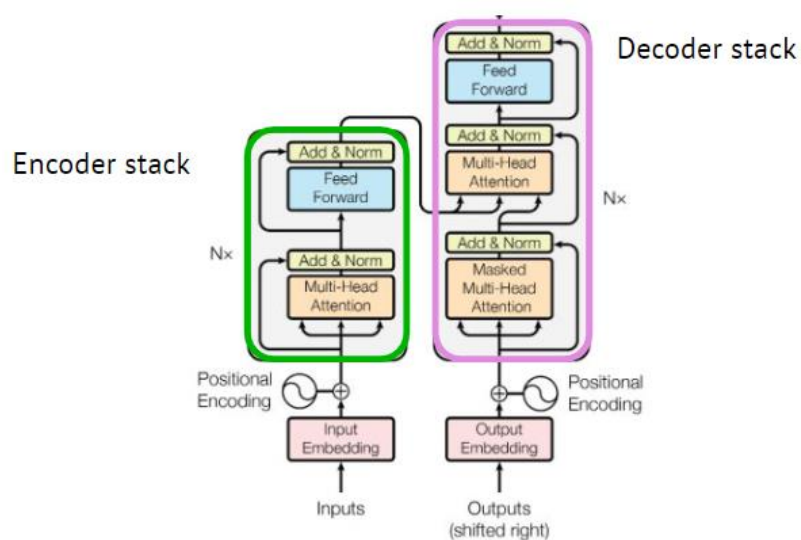


Figure 23 : Structure du Transformer. Source : Attention Is All You Need de Vaswani A. et al.

3.4.2. BERT – Encoder – Bidirectional Attention

Le 24 mai 2019, Jacob Devlin, Ming-Wei Chang, Kenton Lee et Kristina Toutanova propose, dans leur article « BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding », une nouvelle structure **Bidirectional Encoder Representations from Transformers (BERT)**. C'est un modèle pré-entraîné pour la compréhension du langage naturel (NLU). Ainsi il peut donc être facilement finetuned sur un dataset voulu pour réaliser une des tâches du NLP.

Dans le rapport, le modèle a été évalué sur des datasets tels que **SQuAD** (Stanford Question Answering Dataset), **NER** (Named Entity Recognition) et **MNLI** (Multi-genre Natural Language Inference).

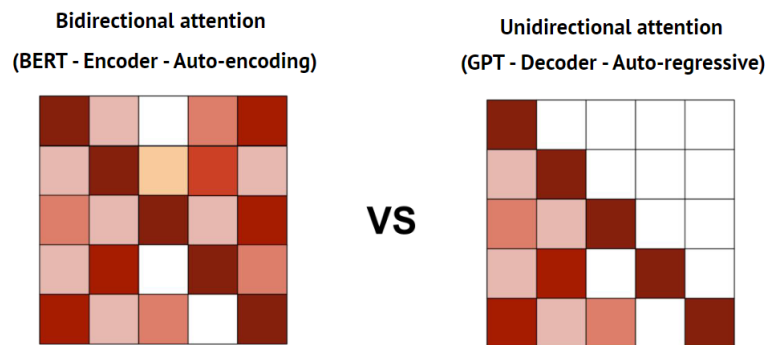


Figure 24 : Attention Bidirectionnelle vs Unidirectionnelle. Source : Vidéo Fidle sur Transformers.

Ensuite, concernant son architecture, il ne possède que des encoders de la forme du Transformer de base. L'avantage de cette structure est qu'il possède une attention bidirectionnelle (figure ci-dessus) qui lui permet de faire des relations avec les mots précédents et suivants ainsi il peut être très précis sur certaines tâches du NLP en lien avec la compréhension du sujet.

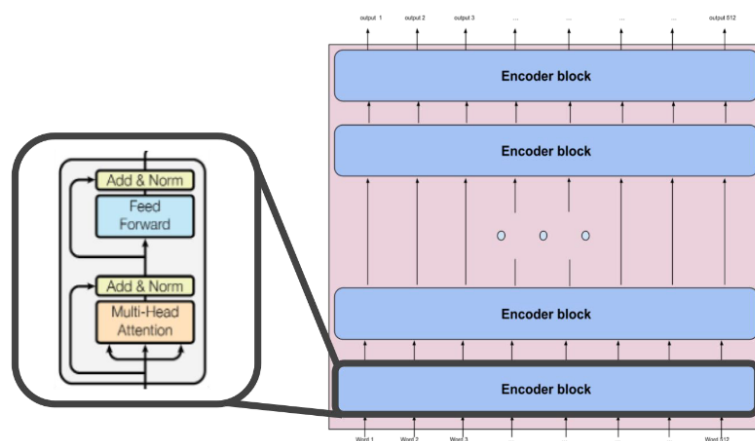


Figure 25 : BERT Encoder. Source : Vidéo Fidle sur Transformers.

3.4.3. GPT – Decoder – Unidirectional Attention

En 2018, l'entreprise OpenAI, spécialisé dans l'IA et le DL, propose leur article « Improving Language Understanding by Generative Pre-Training » avec Generative Pre-Training étant plus connu sous son abréviation **GPT**.

Cette structure, possédant des Masked Multi-Head Attention Transformer et ayant une attention unidirectionnelle, est connu pour sa capacité à générer du texte cohérent et fluide, ce qui le rend adapté à des

tâches telles que la génération de texte, la traduction et le résumé automatique. Il peut également être finement réglé pour d'autres tâches de NLP.

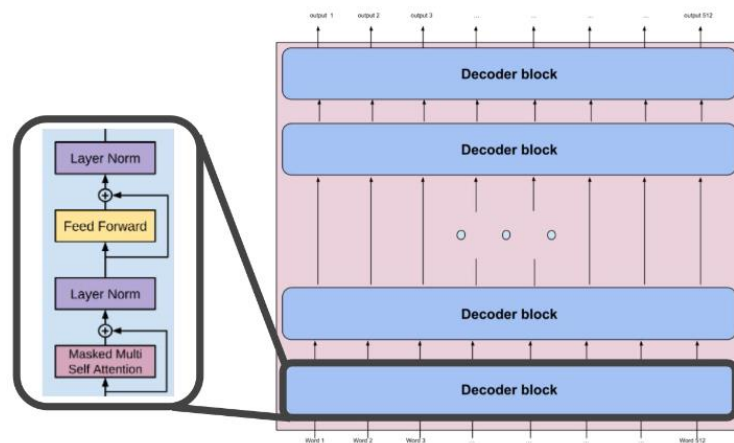


Figure 26 : GPT Decoder. Source : Vidéo Fidle sur Transformers.

E. Récapitulatif

Nous avons vu, dans cette présentation du NLP, les bases et les connaissances à savoir pour débiter dans le domaine du traitement du langage. Certains points n'ont pas pu être expliqués ou détaillés car le monde du NLP est très vaste et est en constante évolution.

Cependant, nous avons, tout de même, abordé les origines du NLP, son histoire, son importance dans le monde actuel et nous avons détaillé la structure omniprésente qui est le Transformer.

Nous allons par la suite, expérimenter dans ce domaine en utilisant Nvidia Toolkit et Python avec Pytorch et la célèbre librairie Transformer d'HuggingFace pour pouvoir tester, affiner et évaluer des modèles.

III. Nvidia et leurs outils de développement

La compagnie Nvidia, initialement leader sur le marché des cartes graphiques, s'est étendue dans le monde l'intelligence artificielle avec la mise en place d'un Nvidia GPU Cloud (**NGC**). L'IA, étant basé sur la puissance de calcul, a permis à Nvidia de se développer dans ce domaine car il possède déjà la puissance du calcul avec leurs GPUs.

NVIDIA GPU Cloud (NGC) est une plateforme cloud qui fournit un large éventail d'outils, de logiciels et de frameworks optimisés pour l'accélération GPU. NGC permet aux développeurs, chercheurs et data scientists d'accéder facilement à des environnements de calcul hautement performants pour leurs travaux liés à l'intelligence artificielle (IA) et au deep learning.

Leur catalogue permet, en collaboration avec Riva Studio, de développer et de déployer des applications de DL en utilisant moins de code. On a notamment suivi une formation pour entraîner un modèle en utilisant TAO Toolkit et une autre pour déployer un modèle sur Riva.

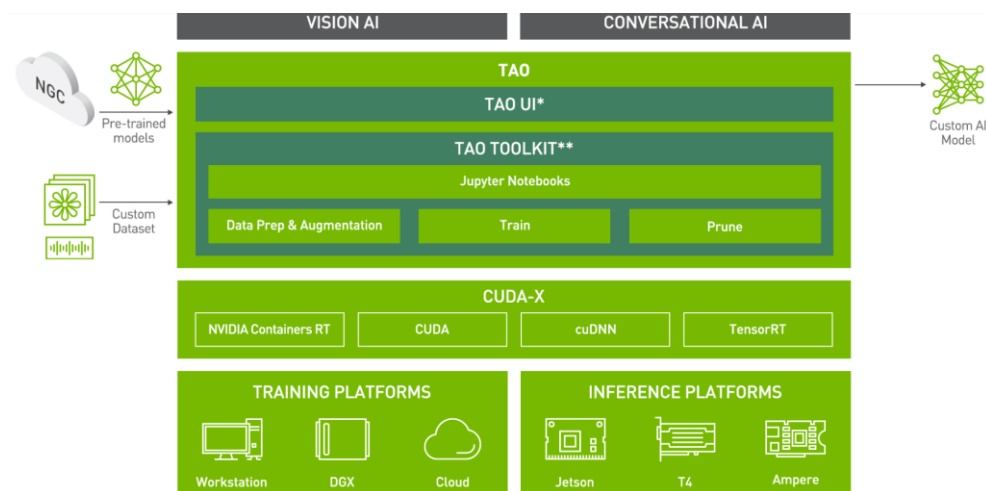


Figure 27 : Outils mis à disposition par Nvidia. Source : Nvidia

NVIDIA® Riva est un Kit de développement logiciel (SDK) de reconnaissance automatique de la parole (ASR) et de synthèse vocale (TTS) accéléré par le GPU qui permet de créer des pipelines d'IA conversationnelle en temps réel entièrement personnalisables et de les déployer dans des clouds, des centres de données, à la périphérie ou sur des appareils embarqués.



Figure 28 : Exemple d'application développée sur Riva. Source : Vidéo Nvidia Developer sur Nvidia Riva.

La création d'application presque sans code possède un inconvénient qui est le nombre de prérequis :

- Création d'un compte sur Nvidia NGC avec l'activation d'une clé.
- Installation des outils de Docker. C'est une *app containerization* qui permet de créer des logiciels sur tout type d'appareils et systèmes d'exploitation.
- Installation des outils Nvidia.

Nous allons donc par la suite voir comment entraîner un modèle avec TAO Toolkit et comment déployer un modèle avec Riva. Le code est disponible sur mon GitHub à cette adresse : <https://github.com/victrolles/DL-NLP-transformers/blob/main/text-classification-training.ipynb>

A. Entraîner un modèle avec TAO Toolkit

1. Task Description

Dans cette partie, nous allons réaliser un modèle d'analyse de sentiment avec le dataset SST-2 pour Stanford Sentiment Treebank. Il possède des données labélisées comme on peut le voir sur l'exemple ci-dessous.

sentence	label
excruciatingly unfunny and pitifully unromantic	0
enriched by an imaginatively mixed cast of antic spirits	1

Figure 29 : Exemples d'élément du dataset. Source : Cours text-classification-training.

2. Download and Preprocess the dataset

L'avantage de TAO Toolkit est qu'il y a très peu de code. En effet, c'est beaucoup de manipulation de fichier. La partie possédant du code est le téléchargement et la préparation du dataset. Ainsi on peut voir ci-dessous un fonction python pour télécharger ce dataset grâce à un lien et pour pouvoir ensuite le stocker à un endroit bien spécifique.

```
import urllib.request
import os

# simple utility function to download the SST-2 dataset
def download_sst2(save_path):

    if not os.path.exists(save_path):
        os.makedirs(save_path)

    # url and filename of the dataset
    download_urls = {
        'https://dl.fbaipublicfiles.com/glue/data/SST-2.zip': 'SST-2.zip',
    }

    for item in download_urls:
        url = item
        file = download_urls[item]
        print('Downloading:', url)
        if os.path.isfile(save_path + '/' + file):
            print('*** Download file already exists, skipping download ***')
        else:
            response = urllib.request.urlopen(url)
            with open(save_path + '/' + file, "wb") as handle:
                handle.write(response.read())

# IMPORTANT Set the following path
DATA_DOWNLOAD_DIR = "/dli/task/data"

# This will download the SST-2 dataset
download_sst2(DATA_DOWNLOAD_DIR)

Downloading: https://dl.fbaipublicfiles.com/glue/data/SST-2.zip

!unzip -o $DATA_DOWNLOAD_DIR/SST-2.zip -d {DATA_DOWNLOAD_DIR}
```

Figure 30 : Download and preprocess the dataset. Source : Cours text-classification-training.

3. Setting TAO Mounts and paths

Il y a ensuite une importante partie de création des dossiers et fichiers. En effet, l'exécuteur TAO utilise docker qui a besoin d'une structure particulière pour fonctionner. On note notamment la configuration de ces dossiers et fichiers dans un fichier « .tao_mounts.json ».

Ensuite la configuration du modèle avec tous les hyperparamètres, les dimensions des batches et d'autres se fait dans les « YAML specification files ». Dans notre cas nous avons téléchargé les paramètres par défaut avec :

```
!tao text_classification download_specs \  
-r $RESULTS_DIR \  
-o $SPECS_DIR
```

Figure 31 : Définition des chemins du TAO. Source : Cours text-classification-training.

4. Dataset Convert

Il est ensuite important de convertir le dataset pour que TAO puisse opérer. Nous devons rentrer les informations suivantes :

- Le nom du dataset
- Le chemin du dataset téléchargé
- Le nouveau chemin du dataset converti

```
# 1. For BERT dataset conversion:  
!tao text_classification dataset_convert \  
-e $SPECS_DIR/dataset_convert.yaml \  
-r $SPECS_DIR/dataset_convert \  
dataset_name=sst2 source_data_dir=$DATA_DIR/SST-2 target_data_dir=$DATA_DIR/sst2
```

Figure 32 : Conversion du dataset. Source : Cours text-classification-training.

5. Training

Après avoir tout configuré, nous avons pu commencer l'entraînement en lançant une commande avec les paramètres suivants :

- -e : Chemin du specification file
- -g : Nombre de GPU
- -k : Clé d'encryptage pour la sauvegarde et le téléchargement du modèle
- -r : Chemin du dossier où sont faites les modifications
- Autres (nombre d'epochs, etc...)

```
# 2. For BERT training on SST-2:  
!tao text_classification train \  
-e $SPECS_DIR/train.yaml \  
-g 1 \  
-k $KEY \  
-r $RESULTS_DIR/train \  
training_ds.file_path=$DATA_DIR/sst2/train.tsv \  
validation_ds.file_path=$DATA_DIR/sst2/dev.tsv \  
model.class_labels.class_labels_file=$DATA_DIR/sst2/label_ids.csv \  
trainer.max_epochs=1
```

Figure 33 : Fonction pour lancer l'entraînement. Source : Cours text-classification-training.

6. Fine-Tuning

On peut ensuite facilement affiner ce modèle avec d'autres datasets. Dans notre cas, nous avons gardé le même dataset SST-2.

```
# 3. For BERT finetuning on SST-2:
!tao text_classification finetune \
  -e $SPECS_DIR/finetune.yaml \
  -g 1 \
  -m $RESULTS_DIR/train/checkpoints/trained-model.tlt \
  -k $KEY \
  -r $RESULTS_DIR/finetune \
  finetuning_ds.file_path=$DATA_DIR/sst2/train.tsv \
  validation_ds.file_path=$DATA_DIR/sst2/dev.tsv \
  trainer.max_epochs=1
```

Figure 34 : Fonction pour affiner le modèle. Source : Cours text-classification-training.

7. Evaluation

L'évaluation du modèle se fait de la même façon. On a exécuté la commande ci-dessous.

```
# 4. For BERT evaluation on SST-2:
!tao text_classification evaluate \
  -e $SPECS_DIR/evaluate.yaml \
  -g 1 \
  -m $RESULTS_DIR/train/checkpoints/trained-model.tlt \
  -k $KEY \
  -r $RESULTS_DIR/evaluate \
  test_ds.file_path=$DATA_DIR/SST-2/dev.tsv \
  test_ds.batch_size=32
```

Figure 35 : Fonction pour évaluer le modèle. Source : Cours text-classification-training.

8. Inference

Pour finir, nous pouvons tester le modèle en ajoutant des phrases dans le infer.yaml. Dans notre cas, nous avons laissé les phrases par défaut.

```
# 5. For BERT inference on user data:
!tao text_classification infer \
  -e $SPECS_DIR/infer.yaml \
  -g 1 \
  -m $RESULTS_DIR/finetune/checkpoints/finetuned-model.tlt \
  -k $KEY \
  -r $RESULTS_DIR/infer
```

Figure 36 : Fonction pour tester le modèle. Source : Cours text-classification-training.

Ce qui nous permet d'obtenir le résultat ci-dessous.

```
[NeMo I 2023-03-07 08:46:02 infer:108] Query: I love exotic science fiction / fantasy movies but this one was very unpleasant to watch
[NeMo I 2023-03-07 08:46:02 infer:109] Predicted label: positive
[NeMo I 2023-03-07 08:46:02 infer:112] Experiment logs saved to '/results/text_classification/infer'
```

Figure 37 : Résultat de l'expérimentation obtenu. Source : Auteur.

9. Exporter sur Riva

Nous avons fini par exporter le modèle en le convertissant en « .riva » avec la commande ci-dessous. Nous allons voir dans la partie suivant comment déployer ce modèle avec Riva.

```
# 8. For export to Riva:
!tao text_classification export \
-e $SPECS_DIR/export.yaml \
-g 1 \
-m $RESULTS_DIR/train/checkpoints/trained-model.tlt \
-k $KEY \
-r $RESULTS_DIR/export_riva \
export_format=RIVA \
export_to=tc-model.riva
```

Figure 38 : Fonction pour exporter sur Riva. Source : Cours text-classification-training.

10. Récapitulatif

Nous avons pu apprendre dans cette formation comment réaliser du NLP avec les outils de Nvidia. C'est une approche assez différente du code python basic. En effet chacun possède ses avantages et ses inconvénients.

B. Déployer un modèle avec Riva

Nous allons apprendre dans cette 2nd partie comment déployer un modèle avec Riva sur le cloud de Nvidia. Nous prendrons cette fois-ci un modèle tout fait. Le notebook est disponible sur mon GitHub à cette adresse : <https://github.com/victrolles/DL-NLP-transformers/blob/main/text-classification-deployment.ipynb>

1. Build Riva

Dans cette première étape, nous définissons le modèle, la tâche à réaliser et les directories.

```
# Get the ServiceMaker docker
! docker pull $RIVA_SM_CONTAINER
# riva-build text_classification
! docker run --rm --gpus 1 -v $MODEL_LOC:/data $RIVA_SM_CONTAINER -- \
| | | riva-build text_classification -f /data/tc-model.rmir:$KEY /data/$MODEL_NAME:$KEY
```

Figure 39 : Build Riva. Source : Cours text-classification-deployment.

2. Deploy Riva

L'outil de déploiement prend en entrée un ou plusieurs fichiers RMIR (Riva Model Intermediate Representation) et un répertoire de dépôt de modèles cible. Il crée une configuration d'ensemble spécifiant le pipeline pour l'exécution et écrit enfin tous ces actifs dans le répertoire du référentiel de modèles de sortie.

```
# Syntax: riva-deploy -f dir-for-rmir/model.rmir:key output-dir-for-repository
! docker run --rm --gpus 1 -v $MODEL_LOC:/data $RIVA_SM_CONTAINER -- \
| | | riva-deploy -f /data/tc-model.rmir:$KEY /data/models
```

Figure 40 : Deploy Riva. Source : Cours text-classification-deployment.

3. Start Riva

Ensuite, nous modifions config.sh pour activer les services Riva appropriés (asr pour le modèle QuartzNet), fournir la clé de cryptage et le chemin vers le référentiel de modèles (`riva\_model\_loc`) généré à l'étape précédente parmi d'autres configurations. Les versions pré-entraînées des modèles spécifiés dans

models_asr/nlp/tts sont récupérées à partir de NGC. Enfin, nous pouvons lancer Riva pour y déployer le modèle.

```
# Execute this cell to copy the solution config.sh into the quickstart directory
! cp solutions/config.sh $RIVA_DIR

# Ensure you have permission to execute these scripts.
! cd $RIVA_DIR && chmod +x ./riva_init.sh && chmod +x ./riva_start.sh

# Run Riva Init. This will fetch the containers/models
# YOU CAN SKIP THIS STEP IF YOU DID RIVA DEPLOY
! cd $RIVA_DIR && ./riva_init.sh config.sh

# Run Riva Start. This will deploy the model(s).
! cd $RIVA_DIR && bash riva_start.sh config.sh
```

Figure 41 : Start Riva. Source : Cours text-classification-deployment.

4. Run Riva

Maintenant que le serveur Riva est prêt, nous pouvons commencer à envoyer des requêtes au serveur. Une api python permet de faire cela comme nous pouvons le voir dans le code ci-dessous.

```

class BertTextClassifyClient(object):
    def __init__(self, grpc_server, model_name):
        # generate the correct model based on precision and whether or not ensemble is used
        print("Using model: {}".format(model_name))

        self.model_name = model_name
        self.channel = grpc.insecure_channel(grpc_server)
        self.riva_nlp = rnlp_srv.RivaLanguageUnderstandingStub(self.channel)

        self.has_bos_eos = False

    # use the text_classification network to return top-1 classes for intents/sequences
    def postprocess_labels_server(self, ct_response):
        results = []

        for i in range(0, len(ct_response.results)):
            intent_str = ct_response.results[i].labels[0].class_name
            intent_conf = ct_response.results[i].labels[0].score

            results.append((intent_str, intent_conf))

        return results

    # accept a list of strings, return a list of tuples ('intent', scores)
    def run(self, input_strings):
        if isinstance(input_strings, str):
            # user probably passed a single string instead of a list/iterable
            input_strings = [input_strings]

        # get intent of the query
        request = rnlp.TextClassRequest()
        request.model.model_name = self.model_name
        for q in input_strings:
            request.text.append(q)
        ct_response = self.riva_nlp.ClassifyText(request)

        return self.postprocess_labels_server(ct_response)

def run_text_classify(server, model, query):
    print("Client app to test text classification on Riva")
    client = BertTextClassifyClient(server, model_name=model)
    result = client.run(query)
    print(result)

```

Figure 42 : Code python. Source : Cours text-classification-deploiement.

Pour finir, on peut voir qu'en exécutant la commande ci-dessous, on reçoit bien les informations du serveur.

```

run_text_classify(server="localhost:50051",
                  model="riva_text_classification_default",
                  query="How is the weather tomorrow?")

Client app to test text classification on Riva
Using model: riva_text_classification_default
[('misty.weather', 0.9970700144767761)]

```

Figure 43 : Résultat de l'expérimentation. Source : Auteur.

C. Conclusion sur Nvidia

Dans cette partie, nous avons pu découvrir les outils Nvidia pour entraîner et déployer un modèle de DL dans le domaine du traitement du langage et ainsi voire une nouvelle méthode qui permet d'éviter le code.

Un des avantages est que les fonctions déjà pré-faites sont dans un 1^{er} temps beaucoup plus optimisé et surtout plus optimisé pour fonctionner sur de multiples GPU.

Cependant, cette méthode restreint la liberté et les possibilités des utilisateurs donc dans le cas de la découverte du NLP et non du déploiement d'une application optimisée de DL, nous allons maintenant découvrir le NLP grâce à la librairie python nommée transformers d'HuggingFace. Puis nous finirons par comprendre comment sont évalués des modèles et nous en évaluerons.

IV. Introduction à HuggingFace et à sa librairie Transformers

HuggingFace est maintenant un incontournable dans la création, ou l'initiation, des modèles de NLP grâce à tous les outils qu'il propose et leur inévitable librairie transformers.

A. Présentation de HuggingFace et sa librairie Transformers

Fondé en 2016 par des Français, **HuggingFace** était initialement une entreprise de chatbot, visant les jeunes, pour pouvoir les faire « rire et sourire » (Clem Delangue, CEO & cofondateur) ; d'où leur célèbre emoji 🤪. C'est maintenant une entreprise américaine proposant de nombreux outils open-source pour réaliser du NLP. Il possède une plateforme en ligne, dont le slogan est « The AI community building the future », qui recense énormément d'outils tels que des modèles de NLP, des datasets, des solutions à des problèmes de NLP, des forums, etc...

Leur popularité est en partie dû à la création de leur librairie python **transformers**, compatible avec Pytorch, TensorFlow et JAX DL, des librairies de Machine Learning (ML) et DL. Elle permet d'utiliser simplement et avec beaucoup de flexibilité des modèles Transformers. En effet, on peut importer des modèles déjà pré-entraînés qu'on peut facilement manipuler et affiner grâce à notre dataset ou des datasets incluse dans la librairie.

B. Fonctionnalités de base

Ensuite grâce à leurs pipelines et leurs fonctions tokenizers, etc., on peut rapidement implémenter un modèle de NLP sur python.

1. Pipeline

Avant tout, tous les pipelines testés sont disponibles sur GitHub à cette adresse :

<https://github.com/victrolles/DL-NLP-transformers#readme>

Le pipeline permet, sans avoir aucune connaissance en NLP, d'implémenter un modèle pour pouvoir répondre aux différents problèmes de ce domaine. J'ai pu tester les principaux comme la classification de texte, la génération de conversation, la génération de suite de texte, trouver le mot manquant, de l'analyse de sentiment, et de la traduction anglais-français. On va voir par la suite plus en détails la génération de texte et l'analyse de sentiment.

On peut voir ci-dessous, le code pour générer du texte grâce à un pipeline, on peut alors importer le bon modèle et les bons paramètres juste en écrivant la tâche à résoudre : « text-generation ». Ainsi, en l'exécutant, on peut voir que le modèle a généré en vert « I can teach you how to move on; I can instruct you how to live your life. ».

```

1  from transformers import pipeline
2
3  sentence = "I can teach you how to"
4
5  generator = pipeline("text-generation")
6
7  result = generator(sentence, max_length=30, do_sample=True)
8
9  print("\nSentence: ", sentence)
10 print(result, "\n")
11
12 # Sentence: I can teach you how to
13 # [{"generated_text": 'I can teach you how to move on; I can instruct you how to live your life.'}]

```

Figure 44 : Pipeline Génération de texte. Source : Auteur.

Ensuite, on peut voir dans un autre exemple, ci-dessous, qu'on peut aussi spécifier le modèle. Dans mon cas, j'ai choisi le modèle « distilbert-base-uncased-finetuned-sst-2-english ». C'est la version la plus optimisée pour de l'analyse de sentiment. En effet, **distilbert** est une version moins lourde et moins complexe de BERT et elle a été entraînée sur le célèbre dataset Stanford Sentiment Treebank, vu précédemment. Ainsi le modèle détecte bien que la sentence 1 est positive à 99% et que la sentence est négative à 98%.

```

1  from transformers import pipeline
2
3  sentence1 = "I love learning new things like NLP"
4  sentence2 = "I hate riding a bike when it's raining and windy"
5
6  # Load the pipeline
7  nlp = pipeline(task="sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
8
9  # Run the pipeline
10 result1, result2 = nlp(sentence1), nlp(sentence2)
11
12 print(sentence1, " : ", result1, "\n", sentence2, " : ", result2)
13
14 # I love learning new things like NLP : [{"label": 'POSITIVE', 'score': 0.9993341565132141}]
15 # I hate riding a bike when it's raining and windy : [{"label": 'NEGATIVE', 'score': 0.9847230911254883}]

```

Figure 45 : Pipeline Analyse de Sentiment. Source : Auteur.

2. Modèle / Tokenizer / Classification

Ensuite, simplement en téléchargeant un modèle et un tokenizer, on peut alors manipuler ce modèle avec **Pytorch** comme s'il s'agissait d'un modèle de DL basique, simple et facile d'utilisation. Vous pouvez retrouver ce code sur GitHub : https://github.com/victrolles/DL-NLP-transformers/blob/main/model_tokenizer.py

De nouveau pour l'analyse de sentiment, grâce aux classes **AutoTokenizer** et **AutoModelForSequenceClassification**, on peut télécharger des tokenizers et modèles déjà entraînés comme on peut l'observer dans mon expérimentation ci-dessous.

```

1  from transformers import AutoTokenizer, AutoModelForSequenceClassification
2  import torch
3  import torch.nn.functional as F
4
5  model_name = "distilbert-base-uncased-finetuned-sst-2-english"
6
7  model = AutoModelForSequenceClassification.from_pretrained(model_name)
8  tokenizer = AutoTokenizer.from_pretrained(model_name)

```

Figure 46 : Import d'un modèle et tokenizer. Source : Auteur.

Ensuite, on peut préparer les données pour le modèle grâce au tokenizer. Dans mon cas, j'ai spécifié le **Padding** pour que toutes les phrases aient la même dimension que la phrase la plus. Concernant la **Truncation**, il faut d'abord savoir que les modèles ont des longueurs maximales en entrée. Pour notre modèle, la limite est de 512 tokens et donc si la limite est dépassée, le tokenizer supprimera les tokens aux extrémités. Aussi, nous devons spécifier `Return_tensors = « pt »` pour Pytorch car la librairie transformers peut fonctionner avec plusieurs librairies de DL (pytorch, tensorflow, keras, MXNet) donc dans notre cas, il faut spécifier que nous travaillons sur Pytorch. Pour chaque phrase, on obtient alors le tensor (~liste) des tokens et le tensor d'attention pour spécifier au modèle s'il doit prendre en compte le token. Ainsi comme la phrase 2 possède plus de tokens que la phrase 1, le padding a ajouté un token vide (id : 0) avec une attention de 0 pour qu'il soit ignoré leur du traitement des données par le modèle.

```
11 inputs = ["I love learning about transformers and deep learning with the Hugging Face library.",
12           "I don't like rugby because it's too violent for me."]
13
14 batch = tokenizer(inputs, padding=True, truncation=True, return_tensors="pt")
15
16 # batch : {'input_ids': tensor([[ 101, 1045, 2293, 4083, 2055, 19081, 1998, 2784, 4083, 2007,
17 #                               1996, 17662, 2227, 3075, 1012, 102, 0],
18 #                               [ 101, 1045, 2123, 1005, 1056, 2066, 4043, 2138, 2009, 1005,
19 #                               1055, 2205, 6355, 2005, 2033, 1012, 102]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0],
20 #                               [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])}
```

Figure 47 : Résultat Tokenizer. Source : Auteur.

Pour finir, comme on peut le voir ci-dessous, on réalise du DL classique. Cette partie n'est pas spécifique au NLP donc nous n'allons détailler. Mais globalement, sans prendre en compte le **Gradient** des tensors avec « `torch.no_grad()` » (pas utilisé car le modèle ne s'entraîne pas et donc nous n'avons besoin de faire de **Back propagation**), nous récupérons la sortie du modèle qui se trouve sous forme de poids. Ensuite on détermine la probabilité des valeurs « Positive » ou « Négative » avec un **Softmax** (somme égale à 1). Puis nous finissons par récupérer la réponse avec la plus grande probabilité. Dans notre cas, on obtient bien une réponse positive pour la 1^{ère} phrase et négative pour la 2nd.

```
26 with torch.no_grad():
27     outputs = model(**batch)
28     predictions = F.softmax(outputs.logits, dim=1)
29     labels = torch.argmax(predictions, dim=1)
30     labels = [model.config.id2label[label_id] for label_id in labels.tolist()]
31
32 # outputs : SequenceClassifierOutput(loss=None, logits=tensor([[ -3.5094,  3.7012],
33 #                               [ 3.4061, -2.8610]]), hidden_states=None, attentions=None)
34
35 # predictions : tensor([[ 7.3817e-04,  9.9926e-01],
36 #                               [ 9.9811e-01,  1.8941e-03]])
37
38 # labels : tensor([1, 0])
39
40 # labels : ['POSITIVE', 'NEGATIVE']
```

Figure 48 : Résultat du modèle. Source : Auteur.

Nous avons maintenant vu quelques possibilités de manipulation avec pytorch et transformers grâce à un exemple.

Après avoir vu les bases, nous allons ensuite faire ce pourquoi HuggingFace a été créé, c'est-à-dire partir d'un modèle déjà existant et pré-entraîné pour comprendre le langage humain et nous allons l'entraîner pour qu'il réponde à une des tâches du NLP.

C. Affinage (Fine-tuning) d'un modèle

Nous allons partir du modèle « bert-base-uncased » en l'important comme vu précédemment. Et nous allons le spécialiser pour qu'il puisse reconnaître du contenu toxique

Pour ce type d'expérimentation, je suis passé sur Google Colab Pro pour être beaucoup rapide. Le notebook utilisé est disponible sur GitHub avec tous les résultats obtenus : https://github.com/victrolles/DL-NLP-transformers/blob/main/FineTune_BERT_Model.ipynb

Nous avons globalement utilisé le même procédé que dans la partie précédente. En effet, j'ai téléchargé les csv de « jigsaw-toxic-comment-classification-challenge » de Kaggle qui prend la forme comme ci-dessous :

	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	0000997932d777bf	Explanation\nWhy the edits made under my usern...	0	0	0	0	0	0
1	000103f0d9cfb60f	D'aww! He matches this background colour I'm s...	0	0	0	0	0	0
2	000113f07ec002fd	Hey man, I'm really not trying to edit war. It...	0	0	0	0	0	0
3	0001b41b1c6bb37e	"\nMore\nI can't make any real suggestions on ...	0	0	0	0	0	0
4	0001d958c54c6e35	You, sir, are my hero. Any chance you remember...	0	0	0	0	0	0

Figure 49 : jigsaw-toxic-comment-classification-challenge. Source : Kaggle

Pour simplifier, j'ai juste récupéré la colonne « toxic ». Ensuite on prépare les données comme vu précédemment.

Maintenant, en important « TrainingArguments, Trainer » de « transformers », on peut assez simplement définir les paramètres comme on peut le voir ci-dessous :

```
# Define Trainer
args = TrainingArguments(
    output_dir="output",
    num_train_epochs=1,
    per_device_train_batch_size=8
)
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)
```

Figure 50 : Trainer. Source : Auteur.

Et pouvoir lancer le processus :

- `trainer.train()` : cette fonction permet de lancer l'entraînement du modèle avec le dataset « train »
- `trainer.evaluate()` : cette fonction permet d'évaluer le modèle avec un autre dataset « eval », ainsi on peut obtenir toutes les statistiques relatives à l'amélioration d'un modèle comme la loss ou l'accuracy.
- `trainer.save_model('path_to_folder')` : cette fonction permet d'enregistrer le modèle après l'avoir entraîné et évalué.

Pour finir, j'ai pu tester avec d'autres phrases, comme on peut le voir ci-dessous, et on remarque dans la réponse finale qu'il a bien réussi à déterminer que les 2 premières phrases sont neutres alors que les 2

suivantes sont toxiques.

```
texts = ["That was good point", "How are you", "go to hell", "I swear you're annoying"]
inputs = tokenizer(texts, padding = True, truncation = True, return_tensors='pt').to('cuda')
with torch.no_grad():
    outputs = model(**inputs)
    predictions = torch.nn.functional.softmax(outputs.logits, dim=-1)
    labels = torch.argmax(predictions, dim=1)
    print(labels)

tensor([0, 0, 1, 1], device='cuda:0')
```

Figure 51 : Résultat de notre modèle affiné. Source : Auteur.

V. Benchmarking de modèles de NLP

A. Définition et objectif du benchmarking de modèle de NLP

Le **benchmarking** est un point de référence standard à partir duquel des mesures sont effectuées. Dans le domaine de l'IA, ces critères de référence sont des ensembles de données collectifs, développés par des entreprises et des groupes universitaires, sur lesquels la communauté s'est mise d'accord pour mesurer les performances des modèles. Par exemple, **SNLI** est une collection de 570 000 paires de phrases anglaises écrites par des humains, étiquetées manuellement pour une classification équilibrée avec les étiquettes implication, contradiction et neutre, qui est utilisée pour mesurer les performances de la tâche d'inférence du langage naturel, NLI.

Comme le monde de l'IA est en pleine émergence, de très nombreux modèles de NLP sont développés chaque jour. Ainsi il est important de pouvoir comparer tous ces modèles sur des données identiques pour pouvoir déterminer et donc garder le meilleur de chacun. De plus, l'histoire a montré que les benchmarks accélèrent le progrès (Hennessy et Patterson, 2011).

B. Développement de benchmarks

1. Histoire et benchmarks existants

Les modèles de langage pré-entraînés tels que BERT ont été révolutionnaires ces dernières années, grâce à l'arrivée des Transformers, pour la compréhension du langage naturel contextualisée et peuvent obtenir d'excellents résultats sur des tâches de traitement du langage naturel. Il est très important de mesurer les performances de ces modèles de langage. Les benchmarks sont devenus plus orientés vers les applications et sont passés de tâches uniques à des tâches multiples. Des exemples de benchmarks multi-tâches sont les benchmarks **GLUE/BLUE**.

Les benchmarks récents célèbres tels que **ImageNet**, **Squad** et **SuperGLUE** ont créé beaucoup d'enthousiasme pour les modèles leaders du classement. Le plus ancien est SPEC, créé en 1988 pour des benchmarks standardisés et des outils d'évaluation des performances et de l'efficacité énergétique des nouveaux systèmes informatiques. Ensuite, **DARPA** et **NIST** ont mis en place des benchmarks précoces pour la reconnaissance vocale et des ensembles de données manuscrites (**MNIST**). Avant les tâches dérivées sur les modèles de langage modernes, la performance d'un modèle de langage était mesurée par la perplexité, l'entropie croisée et

le BPC. Mais maintenant, étant donné que le NLP est principalement axé sur des tâches spécifiques, nous mesurons les performances sur les tâches dérivées telles que l'inférence de langage naturel, la similarité, etc.

2. MLPerf

MLPerf, créé en 2018, est un outil de benchmarking mis en œuvre par un consortium d'experts de l'IA, qui rassemble des organismes universitaires, des laboratoires de recherche et des groupes industriels, dont la mission consiste à **concevoir des benchmarks fiables et efficaces** pour obtenir des évaluations impartiales des performances d'entraînement et d'inférence relatives aux plateformes matérielles, aux solutions logicielles et aux différents services du marché, tout ceci dans des conditions réglementées.

Pour rester à la pointe des dernières tendances de l'industrie, **MLPerf évolue de manière ininterrompue** en réalisant de nouveaux tests à intervalles réguliers et en intégrant de nouvelles charges de travail qui suivent les dernières évolutions de l'IA.

3. MLCommons

Nous avons vu qu'il est important de comparer les résultats des différents modèles pour améliorer la recherche et le développement de technologies d'apprentissage automatique performantes et efficaces.

Cependant, maintenant, avec l'arrivée de gigantesques modèles, une autre préoccupation apparaît : **l'entraînement**. En effet, les modèles comme GPT-3 avec 175Mds de paramètres et BLOOM (Version française de ChatGPT) avec 176Mds de paramètres demandent une puissance de calcul indécemment élevée que seul des supercalculateurs comme celui d'Idris, d'OpenIA, etc... peuvent entraîner. Mais ces supercalculateurs demandent une énorme quantité d'énergie et de temps donc maintenant l'optimisation de l'entraînement de modèles est une des priorités.

Pour cela, MLCommons, anciennement connu sous le nom de MLPerf Training, est lancé en 2020 pour étendre l'impact de MLPerf et cette fois-ci mesurer les performances d'entraînement d'un modèle. Chaque benchmark mesure le temps nécessaire pour former un modèle sur l'ensemble de données spécifié afin d'atteindre l'objectif de qualité demandée.

Ainsi chacun peut déposer ces résultats d'entraînement en suivant un protocole assez stricte disponible sur GitHub : [rules](#).

C. Données mesurées et librairie python

Après avoir bien défini le principe de benchmark et les différents types de benchmarks, nous allons nous intéresser plus en détails dans ce rapport sur l'évaluation d'un entraînement et d'un modèle de NLP qui permet la réalisation de benchmark. Nous allons dans un 1^{er} temps étudier les données concernant l'évaluation du l'entraînement, ensuite du modèle et nous finirons par une application. Nous parlerons des principales statistiques récupérées.

1. Evaluation de l'entraînement d'un modèle

Lors de l'entraînement d'un modèle, les données analysées sont :

- **Le temps d'entraînement total en seconde** : Elle permet de comparer rapidement des résultats entre 2 entraînements. Cependant, ce résultat dépend de très nombreux paramètres comme le nombre d'époques, la taille du batch et toutes les performances de la machine utilisée.
- **Le nombre d'éléments étudié par seconde** : Elle permet de déterminer plus précisément l'optimisation du modèle car on peut savoir si un élément prend du temps à être traité. Cette statistique est notamment très importante aussi lorsqu'on manipule des images car ce sont des données beaucoup plus lourdes et compliquées à traiter que les lignes de textes.
- **Le nombre d'étape étudié par seconde**
- **Le nombre d'opération exécuté** : Cela permet de comparer de manière brute 2 entraînements sans prendre en compte les performances de la machine.
- **La loss** : Elle permet de savoir si le modèle s'entraîne correctement.
- **Le nombre d'époque** : Permet de savoir combien de fois le dataset a été parcourus.

Ainsi, grâce à ces statistiques, nous pouvons plus facilement évaluer un entraînement sur divers aspects pour pouvoir le comparer ensuite avec d'autres méthodes d'entraînement ou d'autres hyperparamètres.

2. Evaluation d'un modèle

Ensuite, pour mesurer les performances d'un modèle de NLP lié à la classification, nous regardons principalement :

- Le taux de réponses corrects (**accuracy**) : C'est la proportion de prédictions correctes par rapport au nombre total de prédictions.
- La précision (**precision**) : C'est la proportion de prédictions prédites comme positives correctes sur toutes les prédictions prédites comme positive. Cette donnée est importante quand on veut se focus sur minimiser les mauvaises prédictions positives, ce qui revient à déterminer à quel point le modèle évite de prédire positif quand ça devrait être négatif.
Par exemple, c'est important pour la classification entre des mails spam ou non. En effet, le modèle doit à tout prix éviter de mettre des mails normaux comme spam alors que l'inverse est moins grave.
- Le rappel (**recall**) : C'est la proportion de prédictions prédites comme positives par rapport au nombre total de prédictions positives. Il permet donc de savoir si le modèle détecte bien les prédictions positives.
- La F-mesure (**f1**) : c'est la moyenne harmonique de la précision et du rappel. Elle permet d'évaluer le rapport entre précision et rappel pour savoir si le modèle est bien équilibré.

Par cela, il est possible de déterminer les performances d'un modèle, et donc pouvoir le comparer avec d'autres.

3. Application avec sklearn.metrics

Le notebook utilisé est disponible sur GitHub avec tous les résultats obtenus : https://github.com/victrolles/DL-NLP-transformers/blob/main/FineTune_BERT_Model.ipynb

Nous allons maintenant voir une simple application. En effet, je suis repartie de mon modèle affiné (fineTuned) dans la partie précédente grâce à Trainer de la librairie transformers. En effet, nous avons entraîné un modèle BERT pour qu'il classe les commentaires entre normal ou toxique.

```
[ ] def compute_metrics(p, epoch=None, logs=None):
    print(type(p))
    pred, labels = p
    pred = np.argmax(pred, axis=1)

    accuracy = accuracy_score(y_true=labels, y_pred=pred)
    recall = recall_score(y_true=labels, y_pred=pred)
    precision = precision_score(y_true=labels, y_pred=pred)
    f1 = f1_score(y_true=labels, y_pred=pred)

    if logs is not None:
        logs[f"epoch_{epoch + 1}_accuracy"] = accuracy
        logs[f"epoch_{epoch + 1}_precision"] = precision
        logs[f"epoch_{epoch + 1}_recall"] = recall
        logs[f"epoch_{epoch + 1}_f1"] = f1

    return {"accuracy": accuracy, "precision": precision, "recall": recall, "f1": f1}

[ ] # Define Trainer
args = TrainingArguments(
    output_dir="output",
    num_train_epochs=1,
    per_device_train_batch_size=8
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    callbacks=[metrics_callback]
)
```

Figure 52 : Compute metrics et Trainer. Source : Auteur.

On définit une fonction compute_metrics qui appelle « accuracy_score, recall_score, precision_score, f1_score » de « **sklearn.metrics** ». Et on peut l'appeler directement dans le Trainer. Ainsi grâce à « trainer.evaluate() », on obtient les performances du modèles. Aussi le « trainer.train() » renvoie automatiquement les performances d'entraînement.

Vous pouvez retrouver, ci-dessous, les performances d'entraînement du modèle avec une seule epoch et ensuite 5. Ici, on remarque que les données sont justes X5 car nous passons de 1 à 5 epochs. Cependant, on remarque que la loss n'est que de 2.5X plus ainsi ce rapport n'est pas proportionnel.

Nombre epochs	1	5	Rapport
Nombre steps	500	2500	5
train_runtime	127	632	4.976377953
train_samples_per_second	31.491	31.64	1.004731511
train_steps_per_second	3.936	3.955	1.004827236
total_flos	1.05E+15	5.26E+15	5.00952381
train_loss	0.186	0.072	2.583333333

Ensuite, voici, ci-dessous, les performances du modèle finale après ces 2 entraînements. On peut effectivement bien remarquer que, après 5 epochs, le modèle possède une meilleure accuracy et un meilleur rapport précision/rappel. Mais, alors que le modèle avec 1 epoch possède une meilleure précision et un moins bon rappel, le modèle avec 5 epochs s'est stabilisé à 0.792. En effet, il fait réussir à trouver le meilleur juste milieu.

nbr_epochs	1	5
eval_accuracy	0.95	0.958
eval_precision	0.823	0.792
eval_recall	0.644	0.792
eval_f1	0.722	0.792

Ainsi, nous avons pu, dans cette partie, comprendre et mettre en pratique l'évaluation d'entraînement et de modèle.

D. Études de cas sur l'évaluation comparative de modèles NLP à l'aide de HuggingFace

Après avoir pris en main le domaine du NLP et les méthodes de benchmarking, nous allons réaliser une petite étude sur des modèles pré-entraînés et finetuned de HuggingFace. Vous pouvez retrouver le code source à cette adresse : <https://github.com/victrolles/DL-NLP-transformers/blob/main/BenchmarkNlpModels.ipynb>

1. Présentation de l'étude de cas

Pour rappel, le principe de benchmarking est de comparer des modèles possédant les mêmes paramètres initiaux tels que le dataset et la machine d'exécution. Nous allons comparer des modèles sur la tâche de classification d'analyse de sentiment : Positif ou Négatif.

- **Dataset** : Les modèles seront comparés sur le dataset connu sst-2 (Stanford Sentiment Treebank 2). Ce dataset possède 67.3k lignes, cependant pour tester les modèles, un échantillon réduit sera suffisant. Ainsi nous prendrons 5% du dataset donc 3367 phrases labélisés.
- **Machine** : Nous utiliserons Google Colab pour exécuter plus rapidement les évaluations des modèles.

Nous avons récupéré 5 modèles de la librairie HuggingFace, dont 2 modèles non finetuned et 3 finetuned :

- **distilbert-base-uncased-finetuned-sst-2-english** : C'est le modèle officiel d'HuggingFace et donc par défaut lorsqu'on souhaite utiliser le pipeline sentimental-analysis dans spécifier de modèle. Il a donc été affiné sur ce dataset sst2.
- **bert-base-uncased** : Pour comparer j'ai choisi de prendre un modèle qui n'a pas été affiné.
- **Ibrahim-Alam/finetuning-roberta-base-on-sst2** : Un particulier a affiné Roberta sur ce dataset sst2.
- **roberta-base** : Ce modèle n'a pas été affiné.
- **Kaludi/Reviews-Sentiment-Analysis** : Ce modèle est intéressant car il permet de faire de l'analyse de sentiment mais il n'a pas été entraîné sur le dataset sst2.

2. Méthode de résolution

La première étape consiste à récupérer la base de données avec la librairie datasets, ne récupérer qu'un petit échantillon et à séparer les phrases des labels en X_test et y_test.

Ensuite pour chaque modèle, il faut le télécharger, ainsi que son tokenizer. Une chose importante est que le modèle ne peut pas prendre en entrée un batch de la dimension du dataset. En effet, dans mes exemples, je n'ai jamais utilisé un batch supérieur à 32 sinon il apparaît un problème de mémoire.

Ainsi on réalise un batch de 32 et sans considérer le gradient, on récupère les prédictions du modèle pour ensuite les concaténer avec les prédictions des autres batches. Ce qui permettra, à la fin, d'obtenir les prédictions sur l'ensemble de nos données d'entrées pour un modèle.

```

# Load the model and the tokenizer
model = AutoModelForSequenceClassification.from_pretrained(name)
tokenizer = AutoTokenizer.from_pretrained(name)
max_length = tokenizer.model_max_length

# Initialize predicted labels
predicted_labels = torch.empty(0, dtype=torch.int64)

# Measure the starting time
start_time = time.process_time()

# Run the model on the all data by group of 32
for i in range(0, len(X_test), batchSize):
    if i%512 == 0:
        print("Step:", i)

    # Prétraiter les données
    batch = tokenizer(X_test[i:i+batchSize], padding=True, truncation=True, max_length=max_length, return_tensors="pt")

    # Faire les prédictions sur les données de test
    with torch.no_grad():
        outputs = model(**batch)
        predictions = F.softmax(outputs.logits, dim=1)
        predicted_labels = torch.cat((predicted_labels, torch.argmax(predictions, dim=1)))

```

Figure 53 : Code python pour charger et évaluer un modèle. Source : Auteur.

J’ai fini par calculer le temps d’exécution et réaliser une classe « AllMetrix » qui récupère les prédictions de tous les modèles et finalement affiche toutes les statistiques.

3. Résultat d’analyse

model	Accuracy	Precision	Recall	F1	Run Time	Step Time	Element Time
distilbert-base-uncased-finetuned-sst-2-english	0.988	0.989	0.989	0.989	228	2.17	0.0678
Ibrahim-Alam/finetuning-roberta-base-on-sst2	0.966	0.964	0.975	0.97	462	4.39	0.137
Kaludi/Reviews-Sentiment-Analysis	0.841	0.797	0.955	0.869	626	5.95	0.186
roberta-base	0.552	0.552	0.989	0.708	458	4.36	0.136
bert-base-uncased	0.55	0.55	1	0.71	449	4.28	0.134

On remarque, dans un 1^{er} temps, que les modèles affinés pour réaliser de la classification par analyse de sentiment ont des bien meilleurs résultats que les modèles pré-entraînés. En effet, les deux modèles pré-entraînés ont une accuracy proche de 50%, ce qui signifie qu’ils agissent de manière aléatoire.

Ensuite concernant l’accuracy, on remarque que des modèles affinés peuvent avoir des résultats assez différents. En effet le « distilbert-base-uncased-finetuned-sst-2-english », très connu sur HuggingFace obtiens les meilleurs résultats. Cependant « Ibrahim-Alam/finetuning-roberta-base-on-sst2 » est assez proche dernière. Cela s’explique, bien évidemment, car ils ont tous les deux été affinés sur ce dataset sst2.

Mais, on observe alors qu’il est plus compliqué d’avoir de bon résultat si le modèle n’a pas été entraîné sur le même dataset. En effet, on obtient 0.841 pour Kaludi/Reviews-Sentiment-Analysis contre 0.966 et 0.988.

De plus, on peut observer que le Recall ne veut rien dire individuellement car pour les deux modèles « roberta-base » et « bert-base-uncased » qui répondent de manière quasi aléatoire ont un rappel respectivement de 0.989 et 1 donc nous pourrions croire que les modèles sont très fort mais finalement non. Pour rappel, le rappel est la proportion de prédictions prédites comme positives par rapport au nombre total de prédictions positives

et comme dans notre cas, beaucoup de réponses négatives sont prédites comme positives alors on obtient un recall très haut.

Aussi, on peut remarquer, que plus un modèle est performant, plus les 4 données sont similaires : comme on peut le voir avec « distilbert-base-uncased-finetuned-sst-2-english » où les résultats sont respectivement 0.988, 0.989, 0.989 et 0.989.

Maintenant, concernant le temps d'exécution, on observe d'assez grandes différences entre les modèles. En effet, le modèle « Kaludi/Reviews-Sentiment-Analysis » est environ 3X plus lent que le plus optimisé « distilbert-base-uncased-finetuned-sst-2-english ». Cela s'explique car distilbert est une version moins lourde de Bert, avec moins de paramètres. Ainsi pour les mêmes performances de machines, on obtient le résultat plus rapidement.

4. Ce que l'on peut retenir de cette analyse

Après cette petite étude de cas, on a pu remarquer plusieurs choses intéressantes :

- **Il ne faut pas prendre les données unes par unes** mais bien toutes ensemble comme on a pu le voir avec le cas du Recall.
- Un modèle qui a été affiné sur le dataset de test obtient de bien meilleur résultat que les autres.
- Un résultat proche de 50% signifie qu'il a donc une réponse aléatoire.
- Même durant la période de test, il est important de prendre en compte le temps d'exécution car on remarque d'assez grande différence entre les modèles. Et ainsi, il ne faut pas le négliger.

VI. Conclusion

A. Récapitulatif du rapport

Nous avons vu, dans un 1^{er} temps, une longue partie théorique pour comprendre tous les éléments de ce domaine du NLP tel que la structure de modèles, les différents modèles existants et le modèle Transformer qui à révolutionner le monde.

Ensuite, à travers des cours Nvidia, nous avons pu apprendre les outils de Nvidia pour développer et déployer, de manière assez simplifiée et différente, des modèles de NLP.

Aussi, nous avons aussi pu découvrir la puissance d'HuggingFace et prendre en main les différentes fonctionnalités de leur librairie transformers. Ainsi nous avons expérimenté et mis en pratique toute la partie théorique vu précédemment.

Pour finir, ayant bien pris connaissance du sujet, nous avons compris comment les modèles et leurs entraînements sont évalués. Et donc nous avons testé cela grâce à la librairie sklearn.metrics et réalisé des benchmarks de quelques modèles d'HuggingFace sur la tâche d'une analyse de sentiment.

B. Mon ressenti sur ce projet

La réalisation de ce rapport a été une expérience hyper enrichissante que j'ai apprécié de A à Z. J'ai vraiment pu explorer et bien prendre en main cet immense domaine du Natural Language Processing.

Ensuite, grâce à des outils efficaces et simple d'utilisation comme les librairie Pytorch et transformers, j'ai pu rapidement expérimenter sur ma propre machine, ce qui a été hyper captivant pour ma part. De plus, HuggingFace est une mine de données très bien organisée avec de très nombreux modèles et datasets, ce qui

permet de tester de nombreuses choses même si nous n'avons pas les compétences requises pour comprendre le modèle.

Enfin, dans un contexte où je vais réaliser un stage de 6 mois à l'IDRIS du CNRS où je vais devoir manipuler, optimiser et benchmarker des modèles de NLP, ce rapport m'a permis de me préparer et ainsi d'être plus à l'aise dans ce domaine.

C. Remerciement

Je tiens à remercier premièrement le professeur **Jaafar Gaber**, qui aura été mon tuteur tout au long du semestre et sans qui je n'aurais pas pu réaliser ce projet de découverte du NLP.

Je tiens aussi à remercier certains membres de l'équipe IDRIS :

- Mon tuteur **Bertrand Cabot** qui m'a donné des liens utiles pour approfondir ce domaine.
- **Hatim Bourfoune** et **Nathan Cassereau** que j'ai pu rencontrer en visioconférence qui m'ont indirectement beaucoup aidé grâce à leur vidéo Fidle sur les Transformers. Ils m'ont aussi donné envie de réaliser ce projet.
- **Jean-luc Parouty** pour les nombreuses autres vidéos Fidle sur les RNN, CNN et données creuses.

VII. Sources

A. Principales sources : vidéos

La plupart de mes connaissances vient des vidéos de HuggingFace avec leur série de vidéos « Hugging Face Course » qui explique tout en détail avec des courtes vidéos.

https://www.youtube.com/watch?v=00GKzGyWFEs&list=PLo2EIpl_JMQvWfQndUesu0nPBAtZ9gP1o&pp=iAQB

Ensuite j'en ai appris beaucoup et plus en détail avec les vidéos de FIDLE de l'IDRIS disponible sur YouTube. J'ai notamment regardé en boucle la vidéo sur les Transformers pour bien comprendre son fonctionnement.

<https://www.youtube.com/watch?v=L3DGgzIbKz4>

Mais aussi d'autres vidéos Fidle tels que celle sur les RNN, les CNN et encore les données creuses.

RNN : <https://www.youtube.com/watch?v=64Yl3K-4FzM>

CNN : <https://www.youtube.com/watch?v=S8gCPOIFyFM>

Données creuses : <https://www.youtube.com/watch?v=zpguGC19UM>

Ensuite j'ai regardé d'autres vidéos :

HuggingFace Crash Course : https://www.youtube.com/watch?v=GSt00_-0ncQ

B. Articles Medium, Datascientest et autres

Définition NLP : <https://datascientest.com/introduction-au-nlp-natural-language-processing>

Définition Transformers : <https://towardsdatascience.com/transformers-89034557de14>

Explication Transformers : <https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452>

Introduction à HuggingFace : https://wandb.ai/int_pb/huggingface/reports/An-Introduction-To-HuggingFace-Transformers-for-NLP--VmIldzoyOTgzMjl5

Benchmarking NLP : <https://rasa.com/blog/benchmarking-language-models/>

C. Thèses et articles de recherche

Rapport Attention is all you Need : <https://arxiv.org/pdf/1706.03762.pdf>

Rapport BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding : <https://arxiv.org/pdf/1810.04805.pdf>

Rapport GLUE: A MULTI-TASK BENCHMARK AND ANALYSIS PLATFORM FOR NATURAL LANGUAGE UNDERSTANDING : <https://arxiv.org/pdf/1804.07461.pdf>

Rapport MLPERF TRAINING BENCHMARK : <https://arxiv.org/pdf/1910.01500.pdf>

D. Sites

Bertrand Cabot m'a proposé de découvrir les benchmarks avec MLCommons : <https://mlcommons.org/en/training-normal-21/>

Goudal Victor

